

✓ Blinkit Data

✓ Import

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display
import plotly.express as px
import plotly.graph_objects as go
```

```
df = pd.read_csv('/content/blinkit_dataset.csv')
display(df.head())
```

	product_id	product_name	category	brand	price	discount_pct	final_price	rating	num_reviews
0	1	Tata Organic Grocery 300	Grocery	Tata	199.78	25	149.84	4.5	10
1	2	Mother Dairy Lite Dairy 275	Dairy	Mother Dairy	44.32	30	31.02	4.0	5
2	3	P&G Classic Personal 439	Personal Care	P&G	501.13	0	501.13	3.7	15
3	4	Dettol Fresh Household 771	Household	Dettol	627.17	0	627.17	3.9	8
4	5	Minute Maid Daily Beverages 264	Beverages	Minute Maid	101.69	15	86.44	4.3	12

5 rows × 10 columns

```
df.tail()
```

	product_id	product_name	category	brand	price	discount_pct	final_price	rat
12995	12996	Minute Maid Classic Beverages 396	Beverages	Minute Maid	219.17	15	186.29	
12996	12997	FreshFarm Organic Fruits 225	Fruits & Vegetables	FreshFarm	348.21	5	330.80	
12997	12998	FreshFarm Original Fruits 349	Fruits & Vegetables	FreshFarm	178.86	10	160.97	
12998	12999	Lizol Lite Household 792	Household	Lizol	435.70	5	413.91	
12999	13000	Dettol Organic Household 987	Household	Dettol	230.42	0	230.42	
5 rows × 25 columns								

```
# Create a working copy
data = df.copy()

print("Original Shape:", data.shape)

Original Shape: (13000, 25)
```

▼ **Data Cleaning**

1. Check duplicates

```
print("Duplicate Rows:", data.duplicated().sum())

Duplicate Rows: 0

data = data.drop_duplicates()

print("Shape After Removing Duplicates:", data.shape)

Shape After Removing Duplicates: (13000, 25)
```

2. Handle missing values

```
print("Missing Values Before Cleaning:")
display(data.isnull().sum())
```

Missing Values Before Cleaning:

	0
product_id	0
product_name	0
category	0
brand	0
price	0
discount_pct	0
final_price	0
rating	0
num_reviews	0
delivery_time_min	0
city	0
seller	0
stock	0
sold_quantity	0
profit_margin_pct	0
is_organic	0
packaging_type	0
weight_g	0
shelf_life_days	0
reorder_level	0
demand_index	0
date_added	0
expiry_date	0
offer_type	6544
delivery_status	0

dtype: int64

```
data["offer_type"] = data["offer_type"].fillna("No Offer")
```

```
print("\nMissing Values After Cleaning:")  
display(data.isnull().sum())
```

Missing Values After Cleaning:

	0
product_id	0
product_name	0
category	0
brand	0
price	0
discount_pct	0
final_price	0
rating	0
num_reviews	0
delivery_time_min	0
city	0
seller	0
stock	0
sold_quantity	0
profit_margin_pct	0
is_organic	0
packaging_type	0
weight_g	0
shelf_life_days	0
reorder_level	0
demand_index	0
date_added	0
expiry_date	0
offer_type	0
delivery_status	0

dtype: int64

3. Convert dates

```
data["date_added"] = pd.to_datetime(data["date_added"])
data["expiry_date"] = pd.to_datetime(data["expiry_date"])

display(data[["date_added", "expiry_date"]].dtypes)
```

0

date_added datetime64[ns]

expiry_date datetime64[ns]

dtype: object

4. Check invalid values

```
print("Price <= 0:", (data["price"] <= 0).sum())
print("Final Price <= 0:", (data["final_price"] <= 0).sum())
print("Rating outside 0-5:", ((data["rating"] < 0) | (data["rating"] > 5)).sum())
print("Discount outside 0-100:", ((data["discount_pct"] < 0) | (data["discount_pc
print("Stock < 0:", (data["stock"] < 0).sum())
print("Sold Quantity < 0:", (data["sold_quantity"] < 0).sum())
print("Delivery Time < 0:", (data["delivery_time_min"] < 0).sum())
```

```
Price <= 0: 0
Final Price <= 0: 0
Rating outside 0-5: 0
Discount outside 0-100: 0
Stock < 0: 0
Sold Quantity < 0: 0
Delivery Time < 0: 0
```

5. Final cleaning check

```
print("\n CLEANED DATASET")

print("Dataset Shape:", data.shape)

print("\nMissing Values:")
print(data.isnull().sum())

print("\nDuplicate Rows:", data.duplicated().sum())

print("\nData Types:")
display(data.dtypes)
```



```
CLEANED DATASET
Dataset Shape: (13000, 25)
```

```
Missing Values:
product_id          0
product_name        0
category            0
brand               0
price               0
discount_pct        0
final_price         0
rating              0
num_reviews         0
delivery_time_min   0
city                0
seller              0
stock               0
sold_quantity       0
profit_margin_pct   0
is_organic          0
packaging_type      0
```

6. Save cleaned dataset

```
data.to_csv("blinkit_cleaned.csv", index=False)
```

```
display("Cleaned dataset saved successfully!")
```

```
date_added          0
date_added          0
date_added          0
offer_type          0
delivery_status     0
dtype: int64
```

EDA

Duplicate Rows: 0

Sales by Category

0

```
plt.figure(figsize=(12, 6))

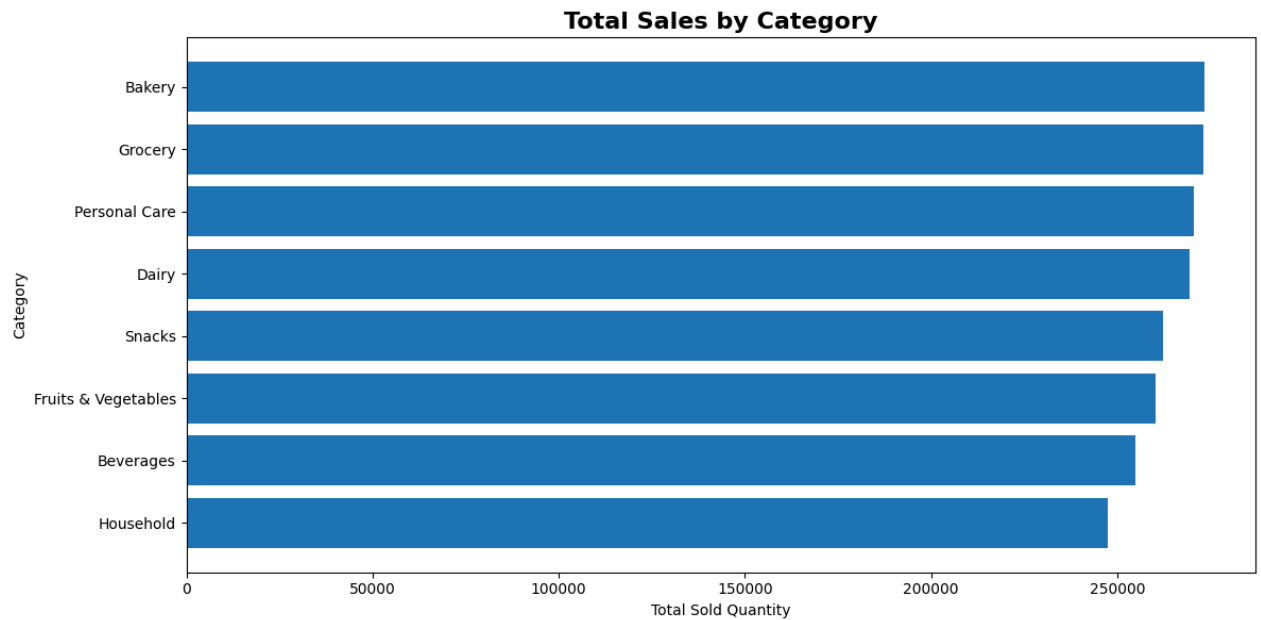
category_sales = (
    data.groupby("category")["sold_quantity"]
    .sum()
    .sort_values(ascending=True)
)

plt.barh(
    category_sales.index,
    category_sales.values
)

plt.title("Total Sales by Category", fontsize=16, fontweight="bold")
plt.xlabel("Total Sold Quantity")
plt.ylabel("Category")

plt.tight_layout()
plt.show()
```

```
stock              int64
sold_quantity      int64
profit_margin_pct   float64
is_organic          bool
```



Sales by City

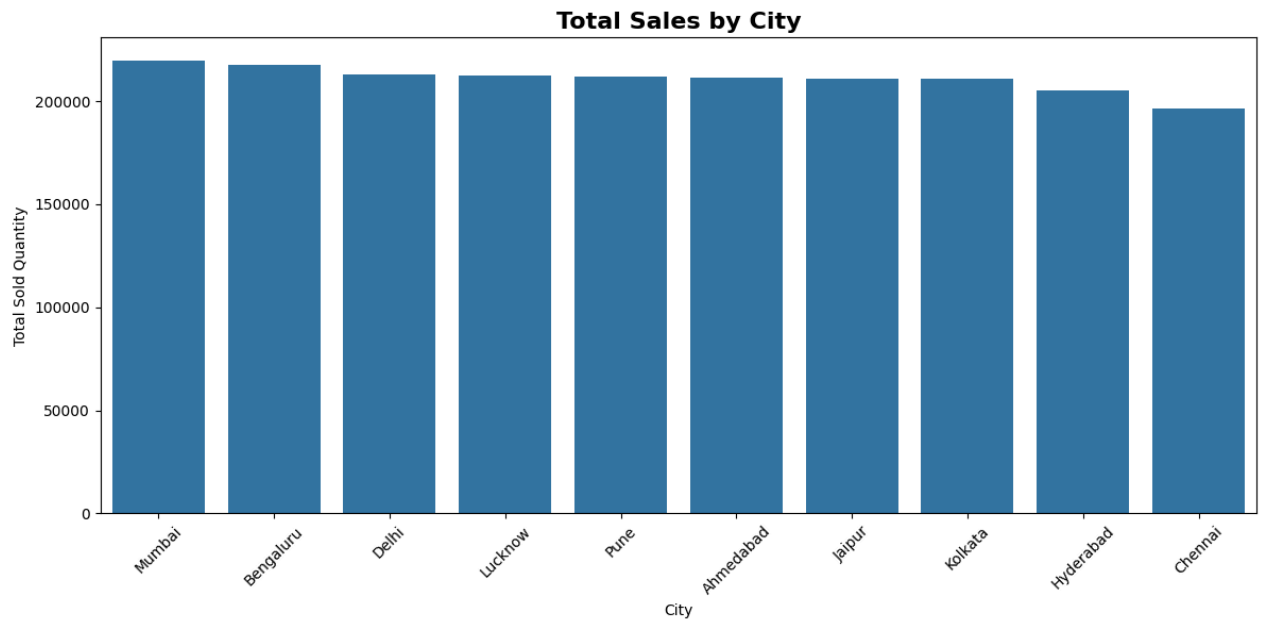
```
city_sales = (
    data.groupby("city")["sold_quantity"]
    .sum()
    .sort_values(ascending=False)
)

plt.figure(figsize=(12, 6))

sns.barplot(
    x=city_sales.index,
    y=city_sales.values
)

plt.title("Total Sales by City", fontsize=16, fontweight="bold")
plt.xlabel("City")
plt.ylabel("Total Sold Quantity")
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```



Price vs Sold Quantity

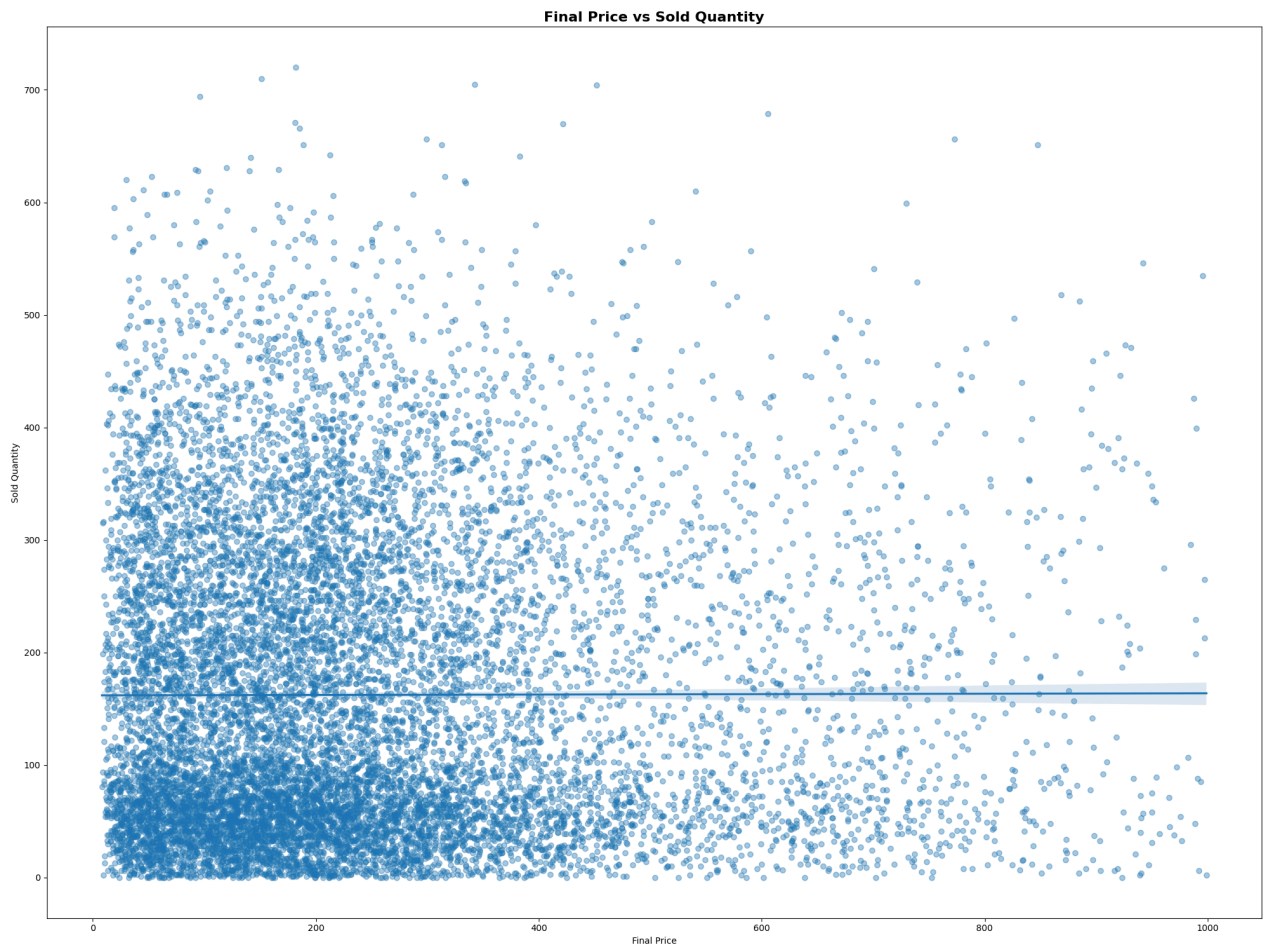
```
plt.figure(figsize=(20, 15))

sns.regplot(
    data=data,
    x="final_price",
    y="sold_quantity",
    scatter_kws={"alpha": 0.4},
    line_kws={}
)

plt.title(
    "Final Price vs Sold Quantity",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Final Price")
plt.ylabel("Sold Quantity")

plt.tight_layout()
plt.show()
```



Demand Index vs Sold Quantity

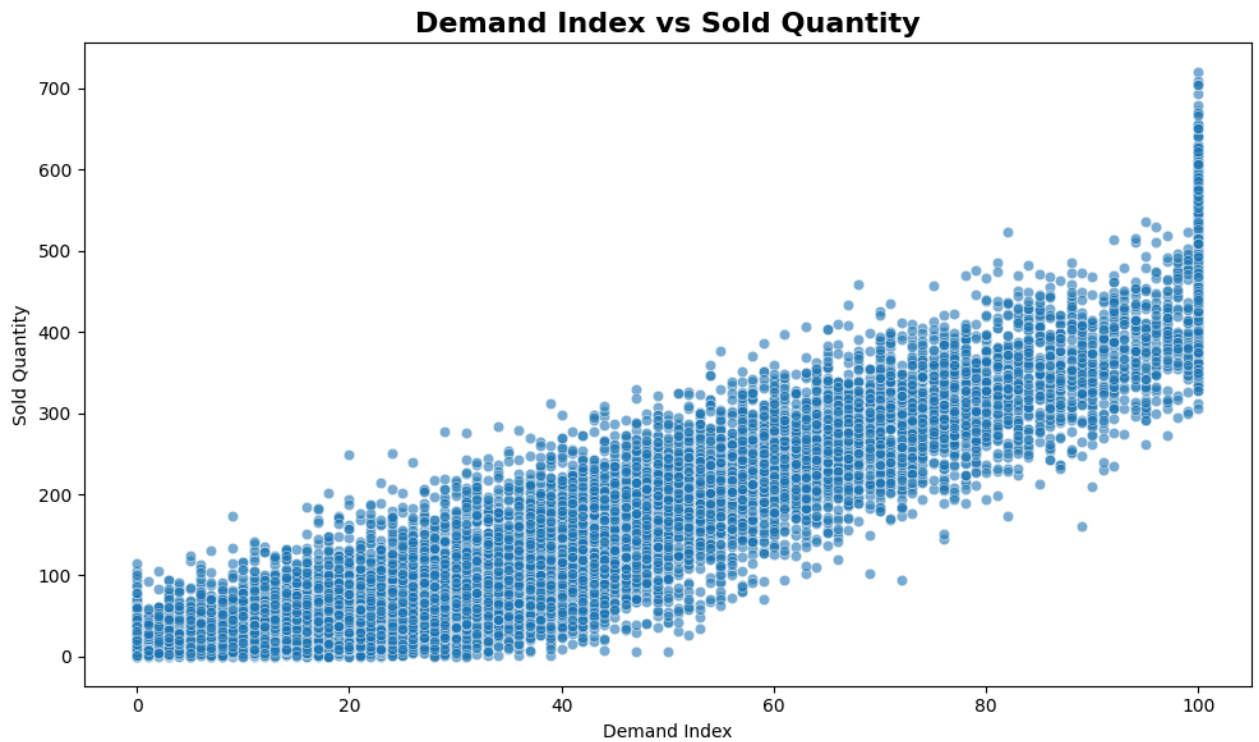
```
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=data,
    x="demand_index",
    y="sold_quantity",
    alpha=0.6
)

plt.title(
    "Demand Index vs Sold Quantity",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Demand Index")
plt.ylabel("Sold Quantity")

plt.tight_layout()
plt.show()
```



Correlation Heatmap

```
numeric_data = data.select_dtypes(include=np.number)

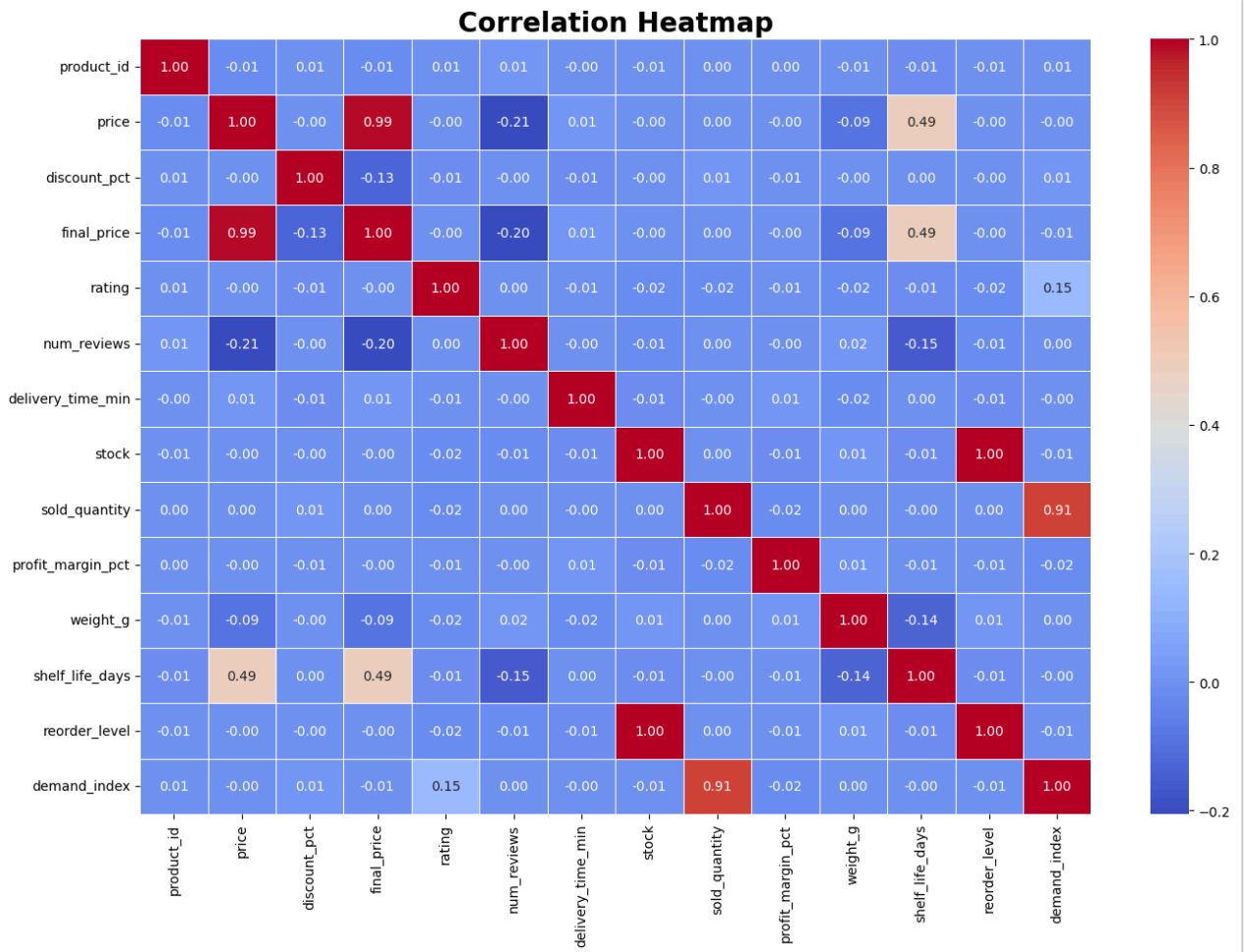
correlation = numeric_data.corr()

plt.figure(figsize=(14, 10))

sns.heatmap(
    correlation,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    linewidths=0.5
)

plt.title(
    "Correlation Heatmap",
    fontsize=20,
    fontweight="bold"
)

plt.tight_layout()
plt.show()
```



Interactive Version

```
fig = px.scatter(
    data,
    x="final_price",
    y="sold_quantity",
    color="category",
    hover_data=[
        "product_name",
        "brand",
        "rating",
        "discount_pct",
        "demand_index"
    ],
    title="Interactive Final Price vs Sold Quantity"
)

fig.show()
```

Interactive Final Price vs Sold Quantity



Final Price × Rating × Sold Quantity

```
fig = px.scatter_3d(
    data,
    x="final_price",
    y="rating",
    z="sold_quantity",
    color="category",
    hover_name="product_name",
    title="3D Price, Rating and Sales Analysis"
)

fig.show()
```

Sales by City

```
city_sales = (
    data.groupby("city")["sold_quantity"]
    .sum()
    .sort_values(ascending=False)
)

print("Sales by City:")
display(city_sales)
```

Sales by City:

	sold_quantity
city	
Mumbai	219930
Bengaluru	217651
Delhi	213243
Lucknow	212665
Pune	212110
Ahmedabad	211695
Jaipur	211135
Kolkata	211046
Hyderabad	205199
Chennai	196527

dtype: int64

```
plt.figure(figsize=(12, 6))

ax = sns.barplot(
    x=city_sales.values,
    y=city_sales.index
)

plt.title(
    "Total Sales by City",
```

```

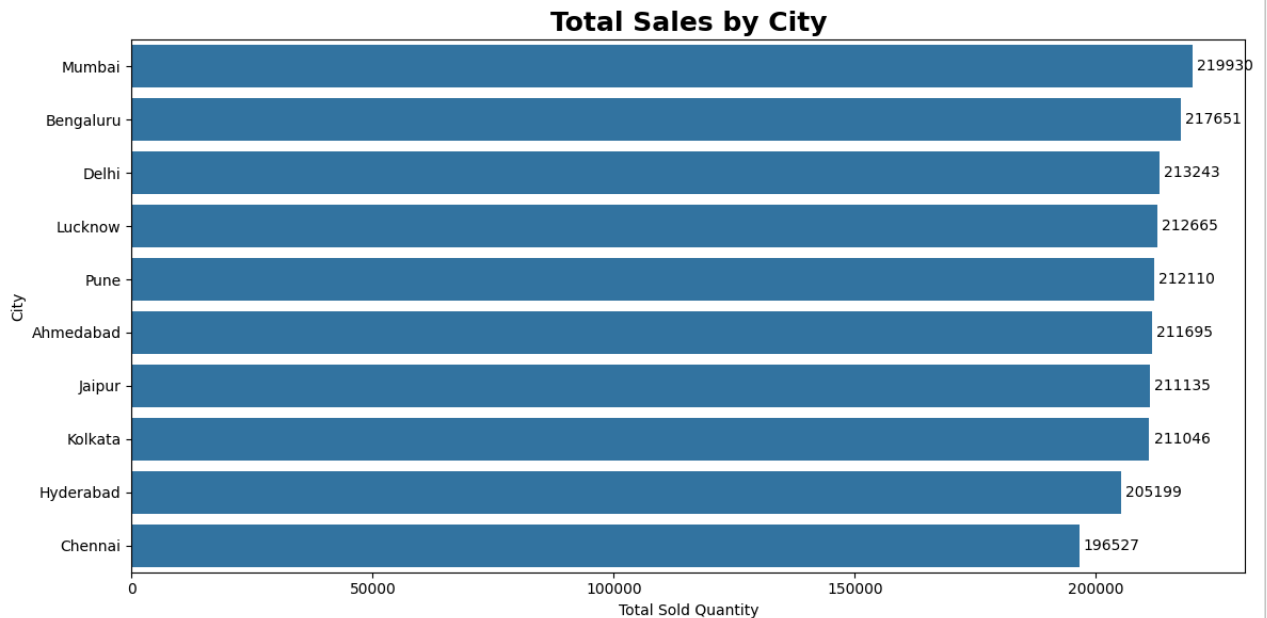
        fontsize=18,
        fontweight="bold"
    )

plt.xlabel("Total Sold Quantity")
plt.ylabel("City")

for container in ax.containers:
    ax.bar_label(container, fmt="%.0f", padding=3)

plt.tight_layout()
plt.show()

```



```

city_sales_df = (
    data.groupby("city", as_index=False)["sold_quantity"]
    .sum()
    .sort_values("sold_quantity", ascending=False)
)

fig = px.bar(
    city_sales_df,
    x="city",
    y="sold_quantity",
    title="Interactive Total Sales by City",
    labels={
        "city": "City",
        "sold_quantity": "Total Sold Quantity"
    },
    text="sold_quantity"
)

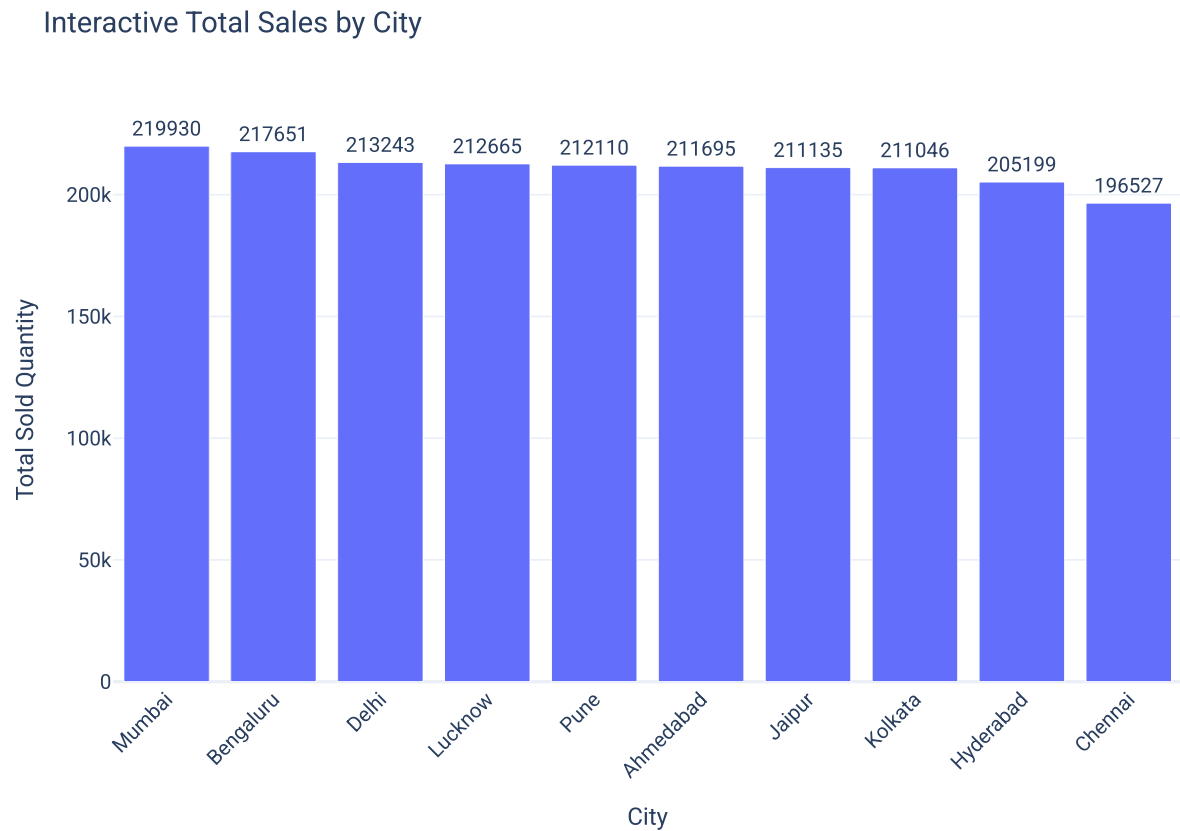
fig.update_traces(textposition="outside")

fig.update_layout(
    xaxis_tickangle=-45,
    template="plotly_white"
)

```

)

fig.show()



Top 10 Brands by Sales

```
top_brands = (  
    data.groupby("brand")["sold_quantity"]  
        .sum()  
        .sort_values(ascending=False)  
        .head(10)  
)  
  
print("Top 10 Brands by Sales:")  
display(top_brands)
```

Top 10 Brands by Sales:

sold_quantity	
brand	
Britannia	162780
DailyGreens	137258
FreshFarm	122908
Modern	90479
BakeHouse	84571
ITC	74645
Aashirvaad	73966
Aavin	73257
Himalaya	70871
Gowardhan	69562

dtype: int64

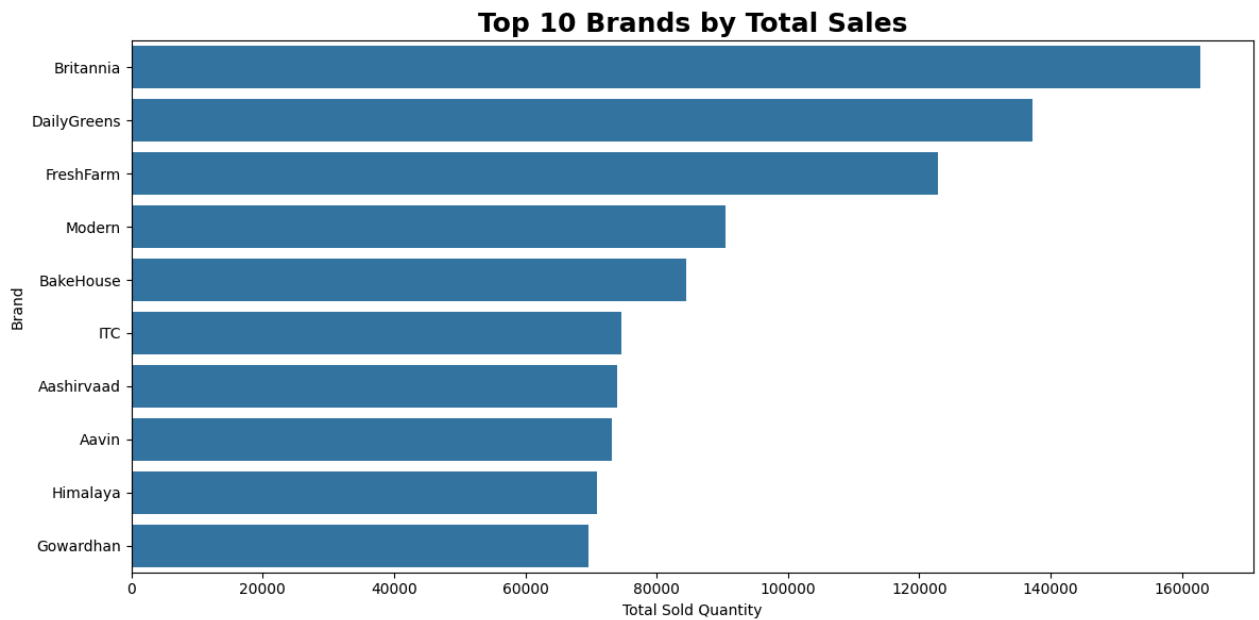
```
plt.figure(figsize=(12, 6))

sns.barplot(
    x=top_brands.values,
    y=top_brands.index
)

plt.title(
    "Top 10 Brands by Total Sales",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel("Total Sold Quantity")
plt.ylabel("Brand")

plt.tight_layout()
plt.show()
```



Delivery Status

```
print("Delivery Status Counts:")
display(data["delivery_status"].value_counts())
```

Delivery Status Counts:

	count
delivery_status	
On-Time	10395
Delayed	2605

dtype: int64

```
plt.figure(figsize=(10, 6))

ax = sns.countplot(
    data=data,
    x="delivery_status"
)

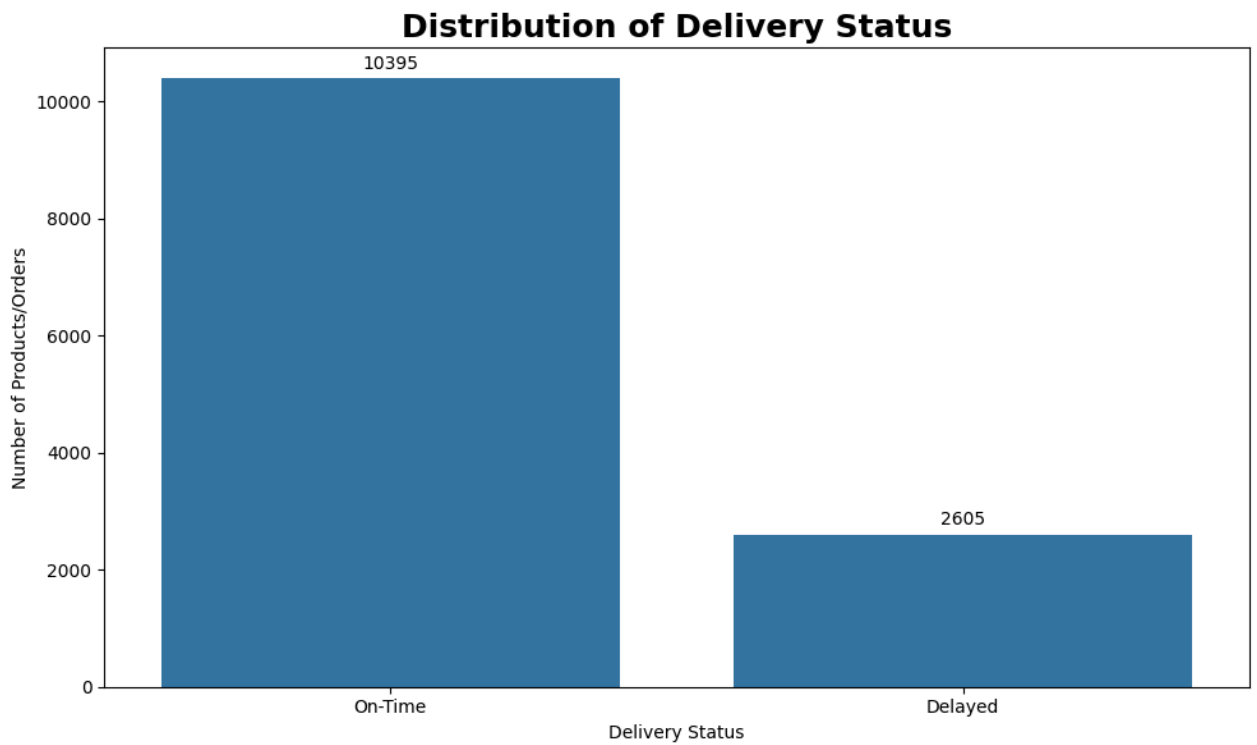
plt.title(
    "Distribution of Delivery Status",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel("Delivery Status")
plt.ylabel("Number of Products/Orders")

for container in ax.containers:
    ax.bar_label(container, padding=3)

plt.tight_layout()
```

```
plt.show()
```



Percentage distribution

```
delivery_percentage = (  
    data["delivery_status"]  
    .value_counts(normalize=True)  
    .mul(100)  
    .round(2)  
)  
  
print("Delivery Status Percentage:")  
display(delivery_percentage)
```

Delivery Status Percentage:

proportion	
delivery_status	
On-Time	79.96
Delayed	20.04

dtype: float64

```
delivery_counts = (  
    data["delivery_status"]  
    .value_counts()  
    .reset_index()  
)
```

```

delivery_counts.columns = ["delivery_status", "count"]

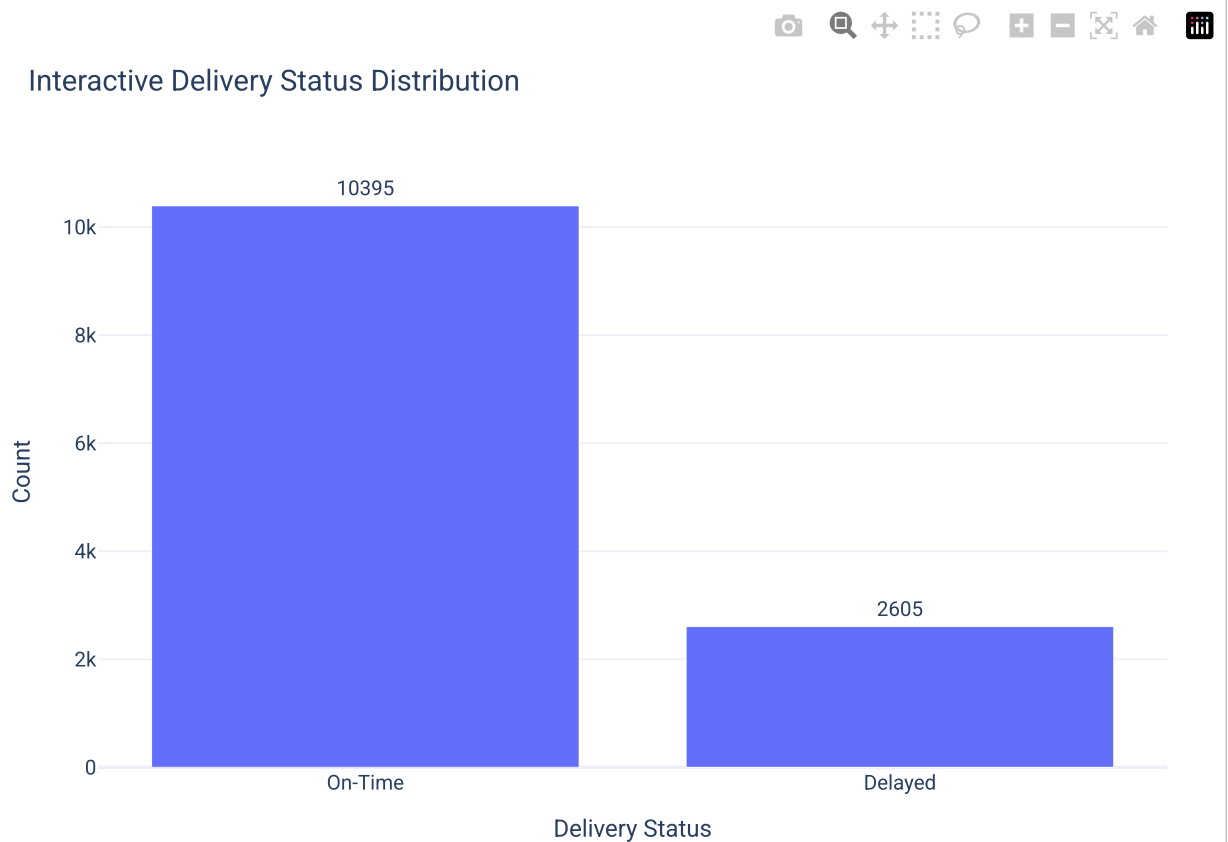
fig = px.bar(
    delivery_counts,
    x="delivery_status",
    y="count",
    text="count",
    title="Interactive Delivery Status Distribution"
)

fig.update_traces(textposition="outside")

fig.update_layout(
    template="plotly_white",
    xaxis_title="Delivery Status",
    yaxis_title="Count"
)

fig.show()

```



Offer Type Distribution

```

print("Offer Type Counts:")
display(data["offer_type"].value_counts())

```

Offer Type Counts:

	count
offer_type	
No Offer	6544
FlatDiscount	2657
Cashback	1895
Buy1Get1	1262
FreeDelivery	642

dtype: int64

```
plt.figure(figsize=(10, 6))

ax = sns.countplot(
    data=data,
    x="offer_type"
)

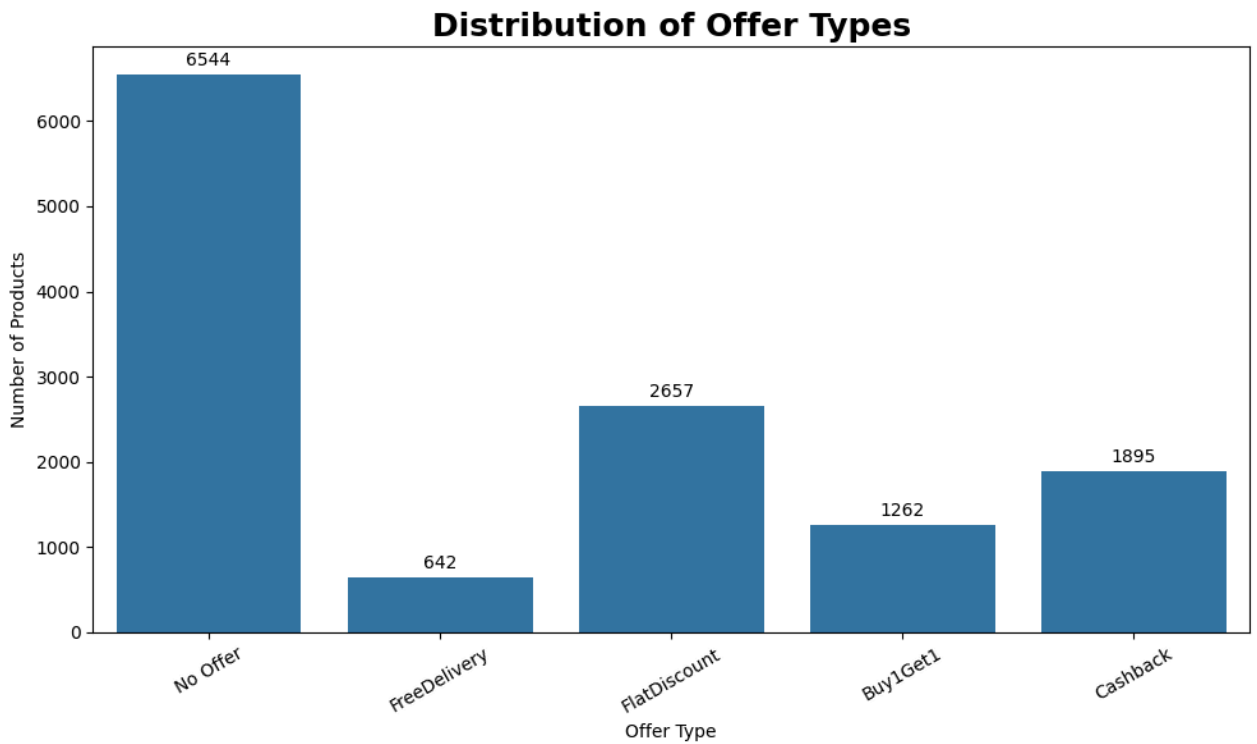
plt.title(
    "Distribution of Offer Types",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel("Offer Type")
plt.ylabel("Number of Products")

plt.xticks(rotation=30)

for container in ax.containers:
    ax.bar_label(container, padding=3)

plt.tight_layout()
plt.show()
```



```
offer_counts = (
    data["offer_type"]
    .value_counts()
    .reset_index()
)

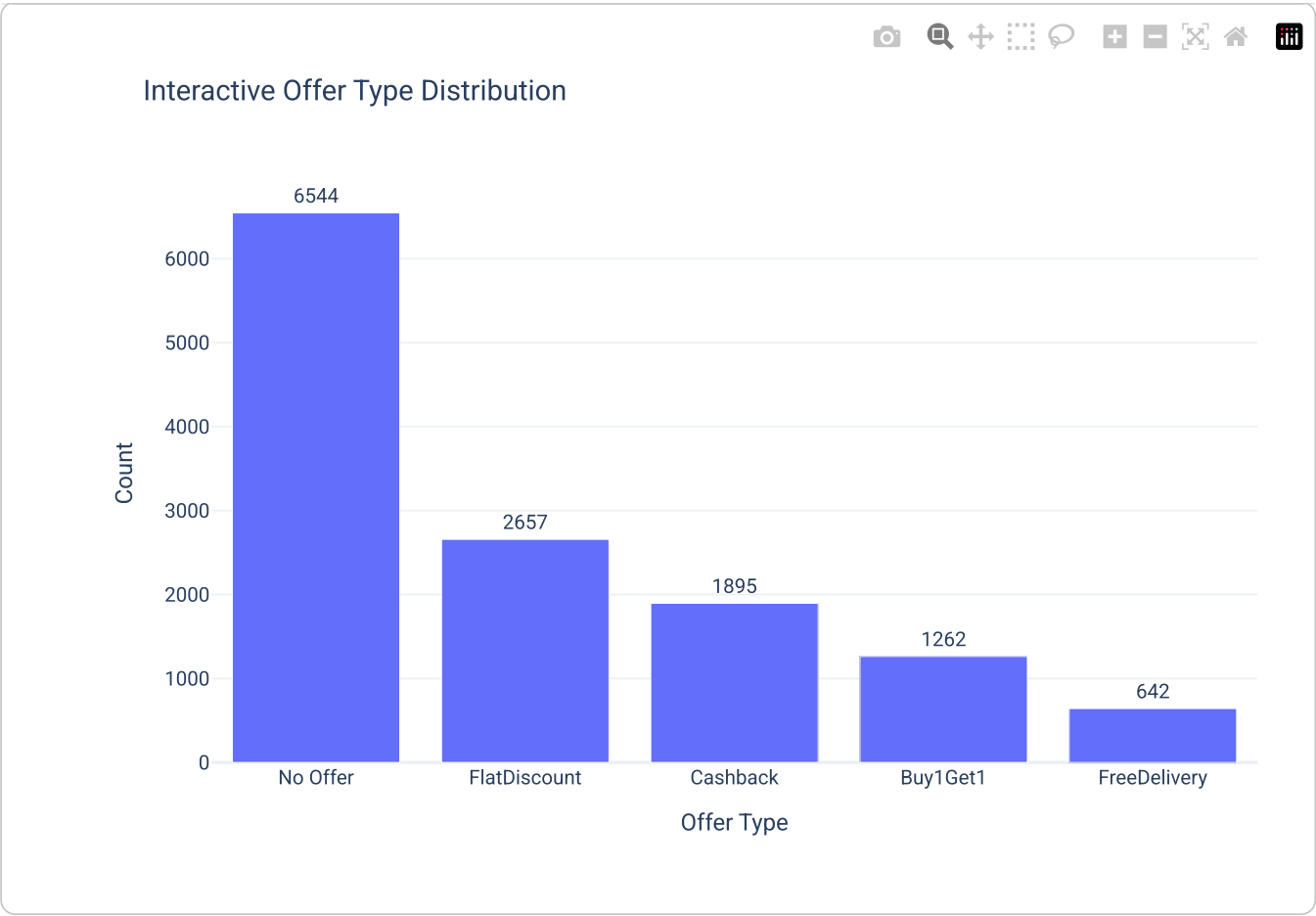
offer_counts.columns = ["offer_type", "count"]

fig = px.bar(
    offer_counts,
    x="offer_type",
    y="count",
    text="count",
    title="Interactive Offer Type Distribution"
)

fig.update_traces(textposition="outside")

fig.update_layout(
    template="plotly_white",
    xaxis_title="Offer Type",
    yaxis_title="Count"
)

fig.show()
```



Price Distribution

```
print("Price Statistics:")
display(data["price"].describe())
```

Price Statistics:

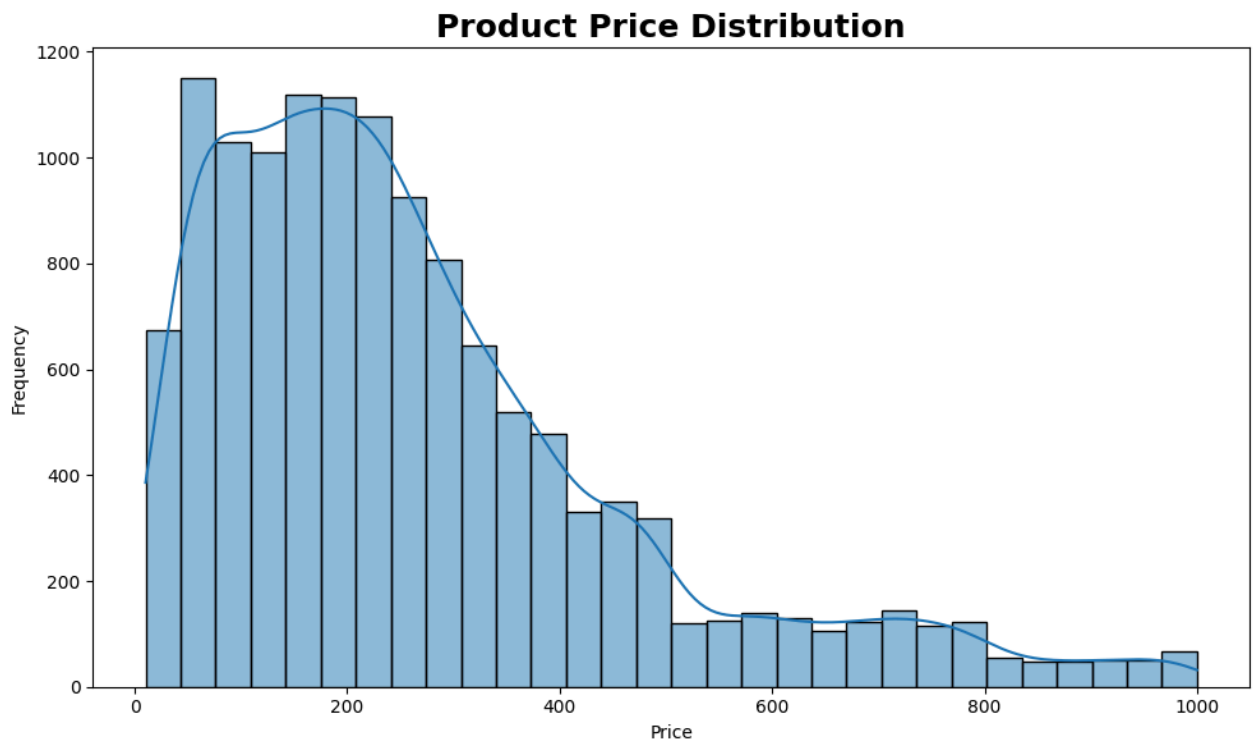
	price
count	13000.000000
mean	267.303356
std	199.665148
min	10.180000
25%	122.972500
50%	220.240000
75%	350.500000
max	999.930000

dtype: float64

```
plt.figure(figsize=(10, 6))

sns.histplot(
    data=data,
    x="price",
```

```
bins=30,  
kde=True  
)  
  
plt.title(  
    "Product Price Distribution",  
    fontsize=18,  
    fontweight="bold"  
)  
  
plt.xlabel("Price")  
plt.ylabel("Frequency")  
  
plt.tight_layout()  
plt.show()
```



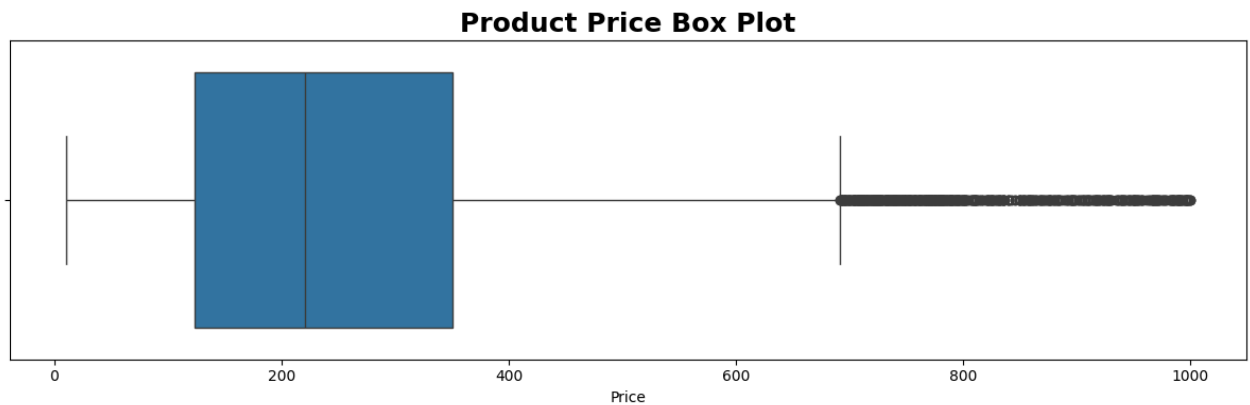
```
plt.figure(figsize=(12, 4))

sns.boxplot(
    x=data["price"]
)

plt.title(
    "Product Price Box Plot",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel("Price")

plt.tight_layout()
plt.show()
```



```
fig = px.histogram(
    data,
    x="price",
    nbins=30,
    marginal="box",
    title="Interactive Product Price Distribution",
    labels={"price": "Price"}
)

fig.update_layout(
    template="plotly_white"
)

fig.show()
```



Rating Distribution

```
display("Rating Statistics:")
display(data["rating"].describe())
```

'Rating Statistics:'

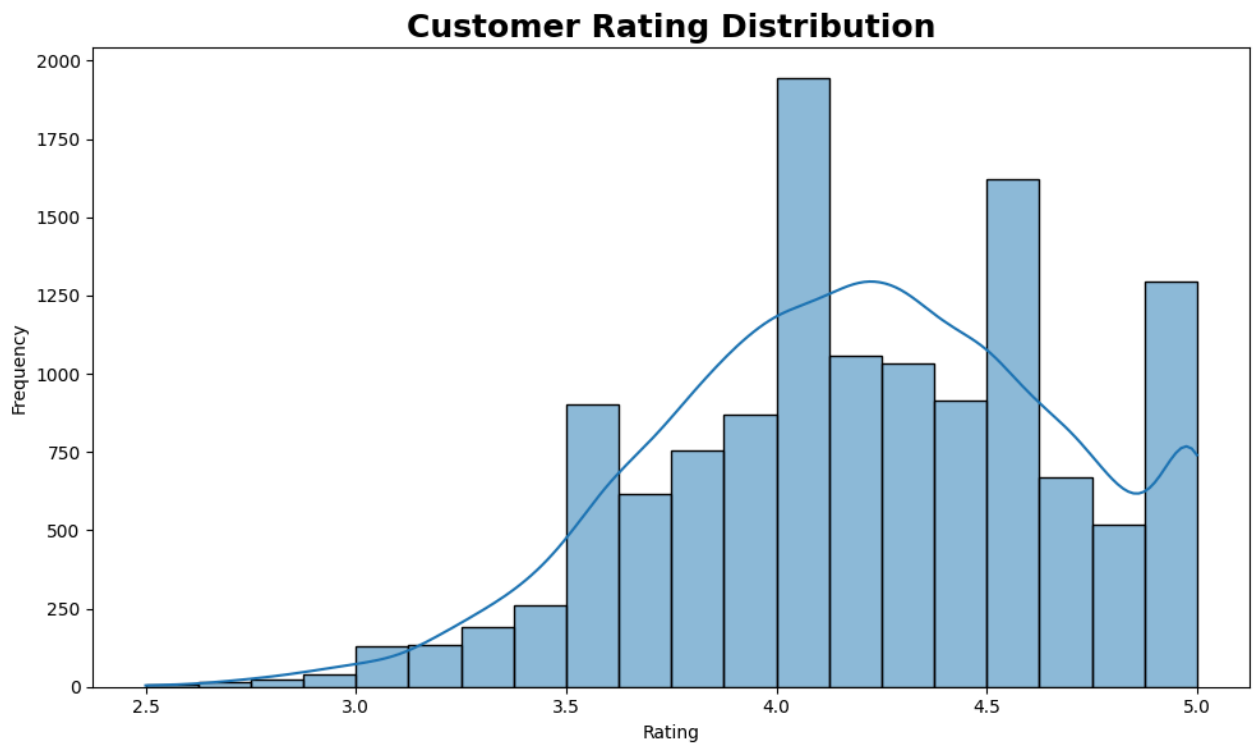
	rating
count	13000.000000
mean	4.196931
std	0.475863
min	2.500000
25%	3.900000
50%	4.200000
75%	4.500000
max	5.000000

dtype: float64

```
plt.figure(figsize=(10, 6))

sns.histplot(
    data=data,
    x="rating",
```

```
    bins=20,  
    kde=True  
)  
  
plt.title(  
    "Customer Rating Distribution",  
    fontsize=18,  
    fontweight="bold"  
)  
  
plt.xlabel("Rating")  
plt.ylabel("Frequency")  
  
plt.tight_layout()  
plt.show()
```



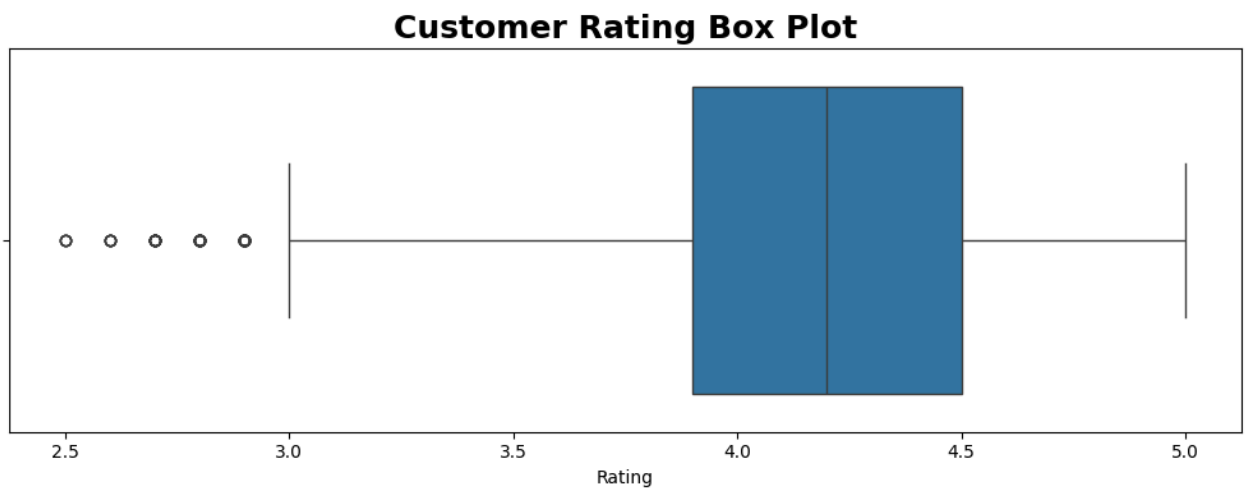
```
plt.figure(figsize=(10, 4))

sns.boxplot(
    x=data["rating"]
)

plt.title(
    "Customer Rating Box Plot",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel("Rating")

plt.tight_layout()
plt.show()
```



```
fig = px.histogram(
    data,
    x="rating",
    nbins=20,
    marginal="box",
    title="Interactive Customer Rating Distribution",
    labels={"rating": "Customer Rating"}
)

fig.update_layout(
    template="plotly_white"
)

fig.show()
```



Sold Quantity Distribution

```
print("Sold Quantity Statistics:")
display(data["sold_quantity"].describe())
```

Sold Quantity Statistics:

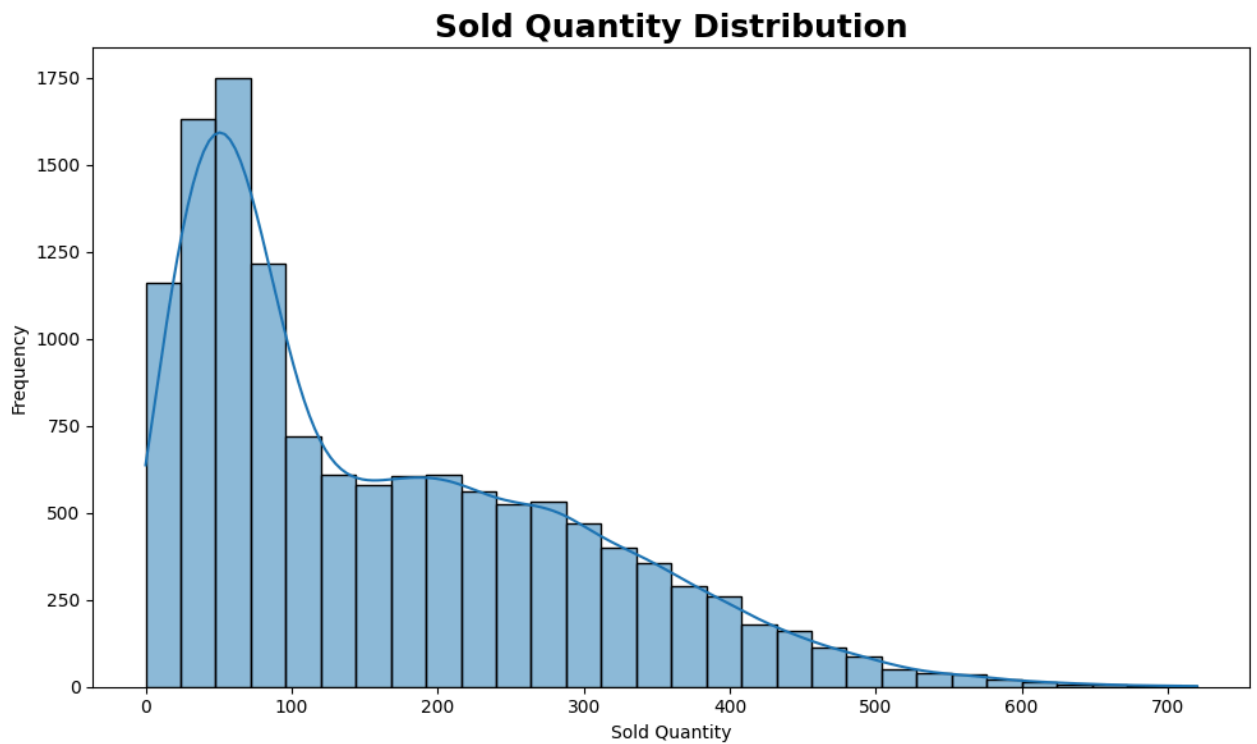
sold_quantity	
count	13000.000000
mean	162.400077
std	132.742425
min	0.000000
25%	54.000000
50%	120.000000
75%	253.000000
max	720.000000

dtype: float64

```
plt.figure(figsize=(10, 6))

sns.histplot(
    data=data,
    x="sold_quantity",
```

```
bins=30,  
kde=True  
)  
  
plt.title(  
    "Sold Quantity Distribution",  
    fontsize=18,  
    fontweight="bold"  
)  
  
plt.xlabel("Sold Quantity")  
plt.ylabel("Frequency")  
  
plt.tight_layout()  
plt.show()
```



```

plt.figure(figsize=(12, 4))

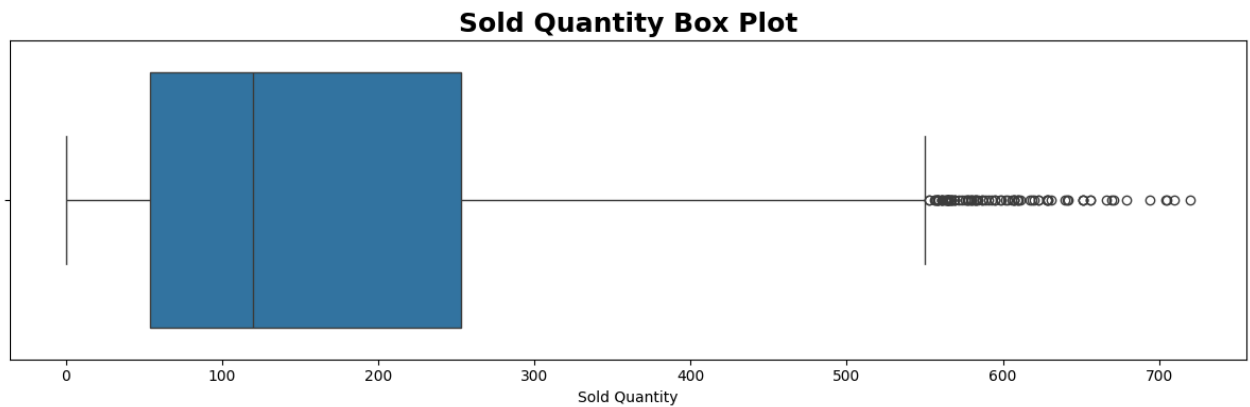
sns.boxplot(
    x=data["sold_quantity"]
)

plt.title(
    "Sold Quantity Box Plot",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel("Sold Quantity")

plt.tight_layout()
plt.show()

```



```

fig = px.histogram(
    data,
    x="sold_quantity",
    nbins=30,
    marginal="box",
    title="Interactive Sold Quantity Distribution",
    labels={"sold_quantity": "Sold Quantity"}
)

fig.update_layout(
    template="plotly_white"
)

fig.show()

```



Demand Index Distribution

```
print("Demand Index Statistics:")
display(data["demand_index"].describe())
```

Demand Index Statistics:

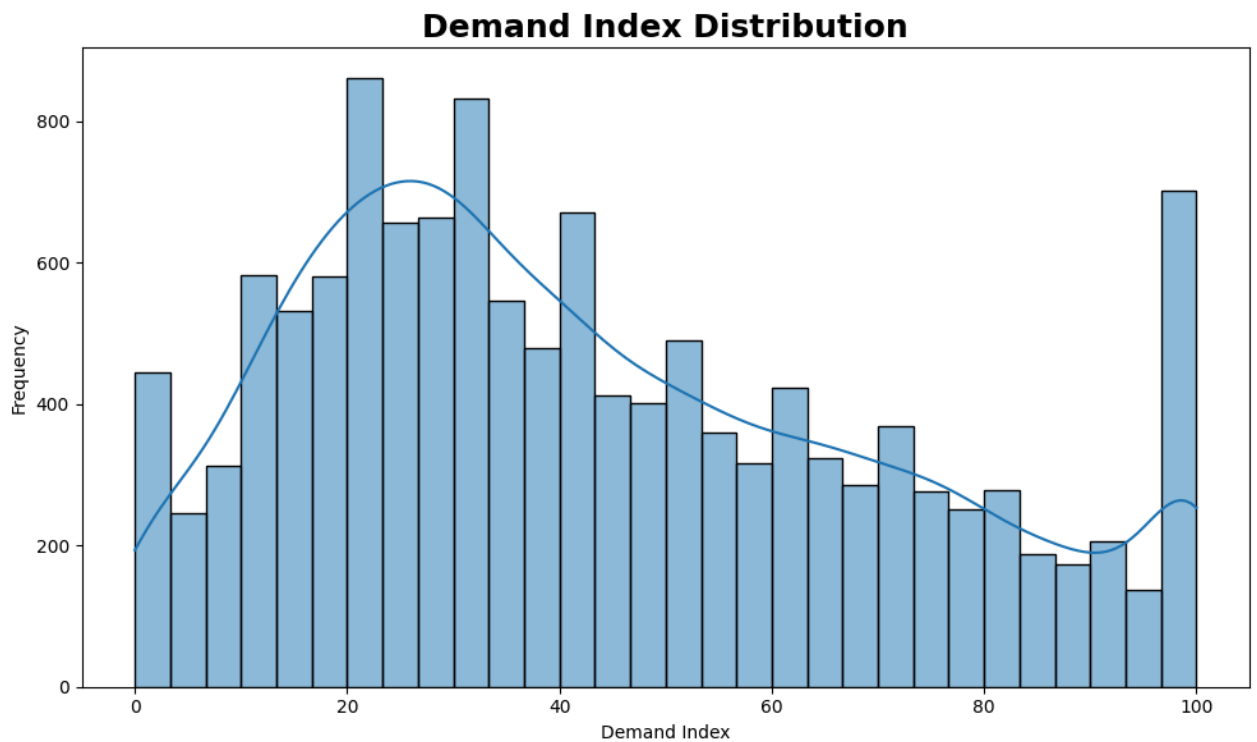
	demand_index
count	13000.000000
mean	43.380462
std	27.076935
min	0.000000
25%	22.000000
50%	38.000000
75%	63.000000
max	100.000000

dtype: float64

```
plt.figure(figsize=(10, 6))

sns.histplot(
    data=data,
    x="demand_index",
```

```
bins=30,  
kde=True  
)  
  
plt.title(  
    "Demand Index Distribution",  
    fontsize=18,  
    fontweight="bold"  
)  
  
plt.xlabel("Demand Index")  
plt.ylabel("Frequency")  
  
plt.tight_layout()  
plt.show()
```



```

plt.figure(figsize=(12, 4))

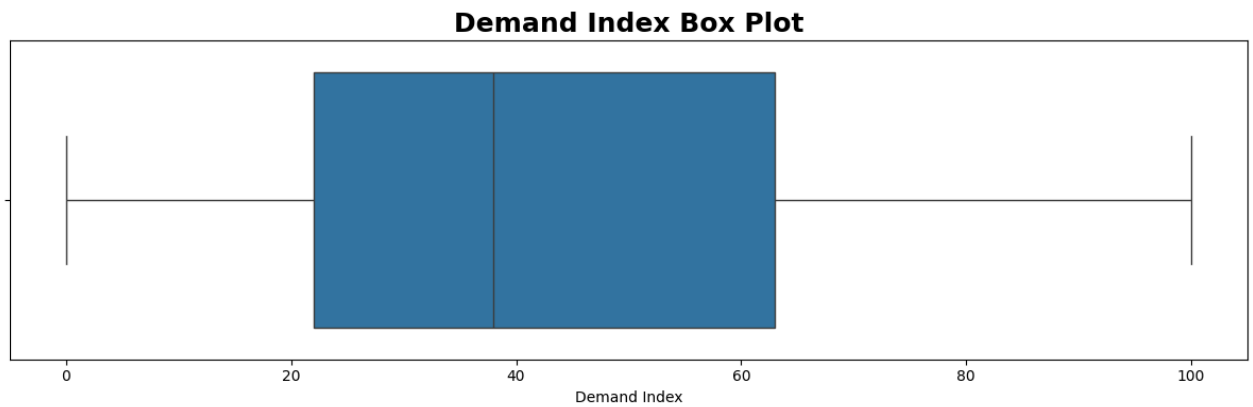
sns.boxplot(
    x=data["demand_index"]
)

plt.title(
    "Demand Index Box Plot",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel("Demand Index")

plt.tight_layout()
plt.show()

```



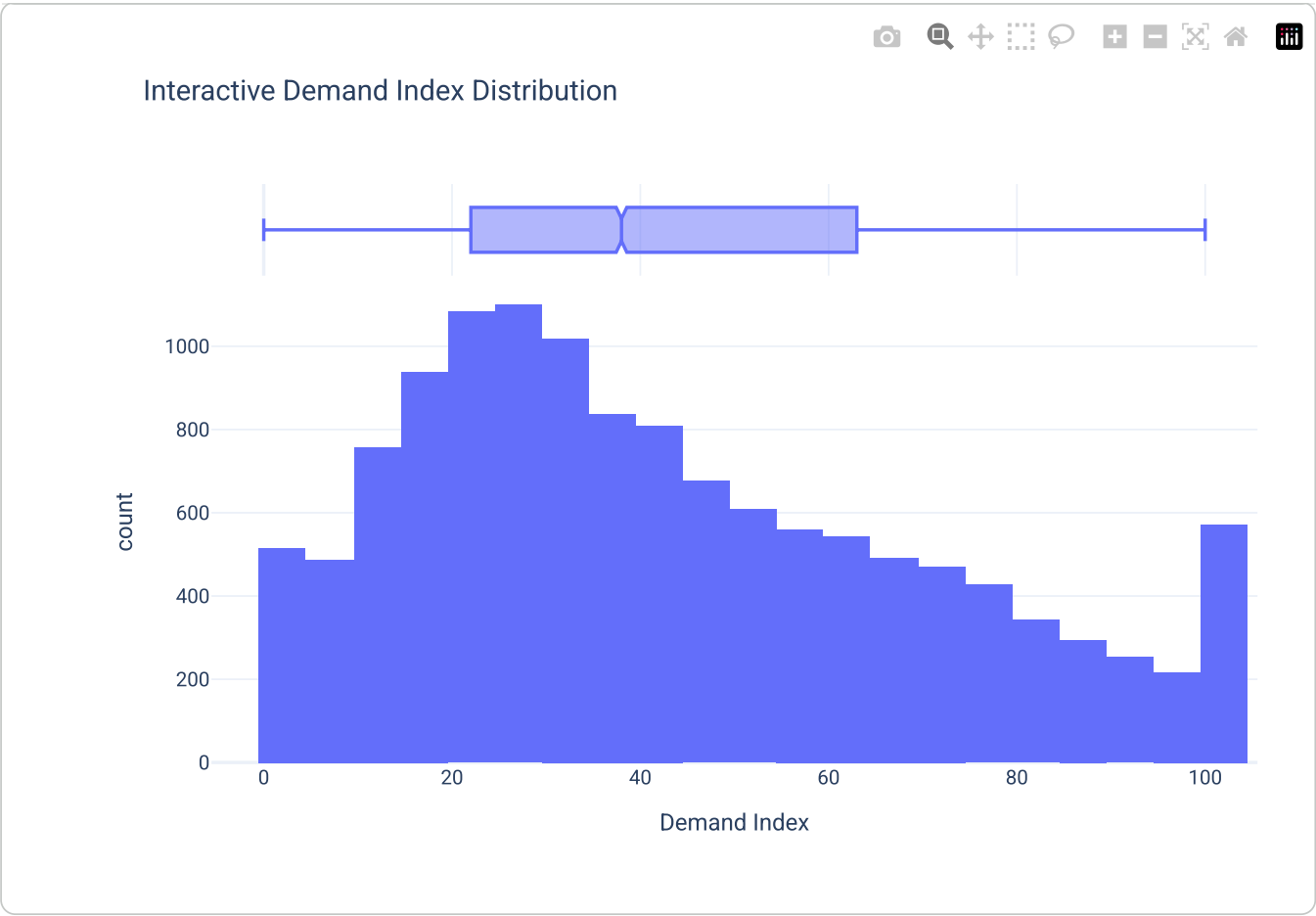
```

fig = px.histogram(
    data,
    x="demand_index",
    nbins=30,
    marginal="box",
    title="Interactive Demand Index Distribution",
    labels={"demand_index": "Demand Index"}
)

fig.update_layout(
    template="plotly_white"
)

fig.show()

```



Discount Distribution

```
print("Discount Statistics:")
display(data["discount_pct"].describe())
```

Discount Statistics:

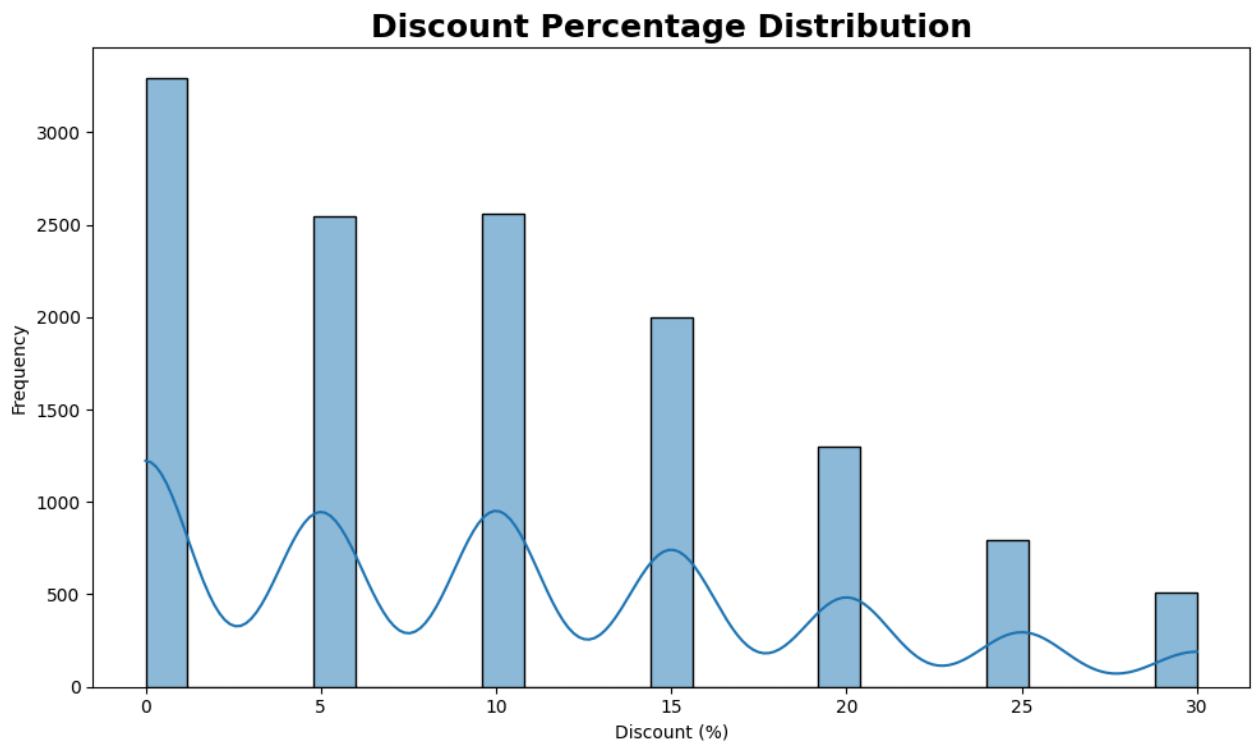
	discount_pct
count	13000.000000
mean	9.959231
std	8.575017
min	0.000000
25%	0.000000
50%	10.000000
75%	15.000000
max	30.000000

dtype: float64

```
plt.figure(figsize=(10, 6))

sns.histplot(
    data=data,
    x="discount_pct",
```

```
    bins=25,  
    kde=True  
)  
  
plt.title(  
    "Discount Percentage Distribution",  
    fontsize=18,  
    fontweight="bold"  
)  
  
plt.xlabel("Discount (%)")  
plt.ylabel("Frequency")  
  
plt.tight_layout()  
plt.show()
```



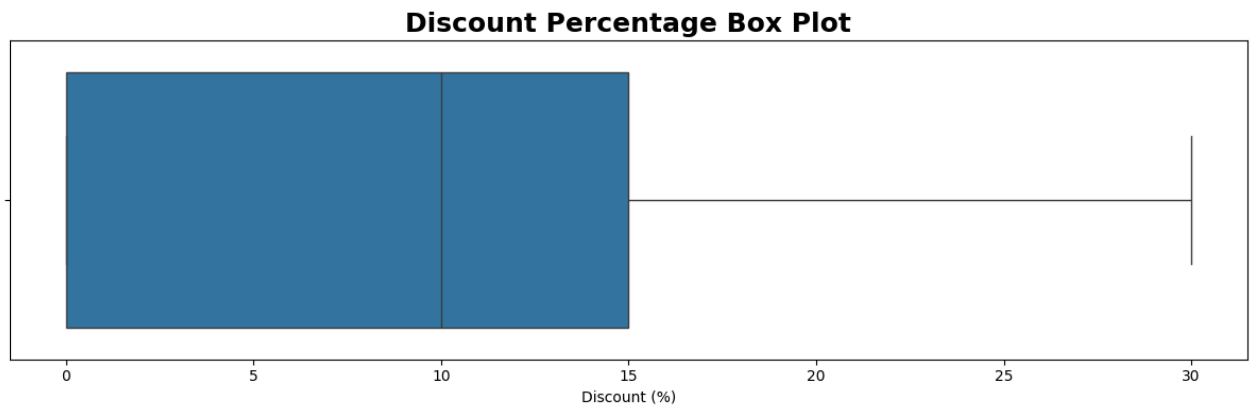
```
plt.figure(figsize=(12, 4))

sns.boxplot(
    x=data["discount_pct"]
)

plt.title(
    "Discount Percentage Box Plot",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel("Discount (%)")

plt.tight_layout()
plt.show()
```



Price vs Sold Quantity

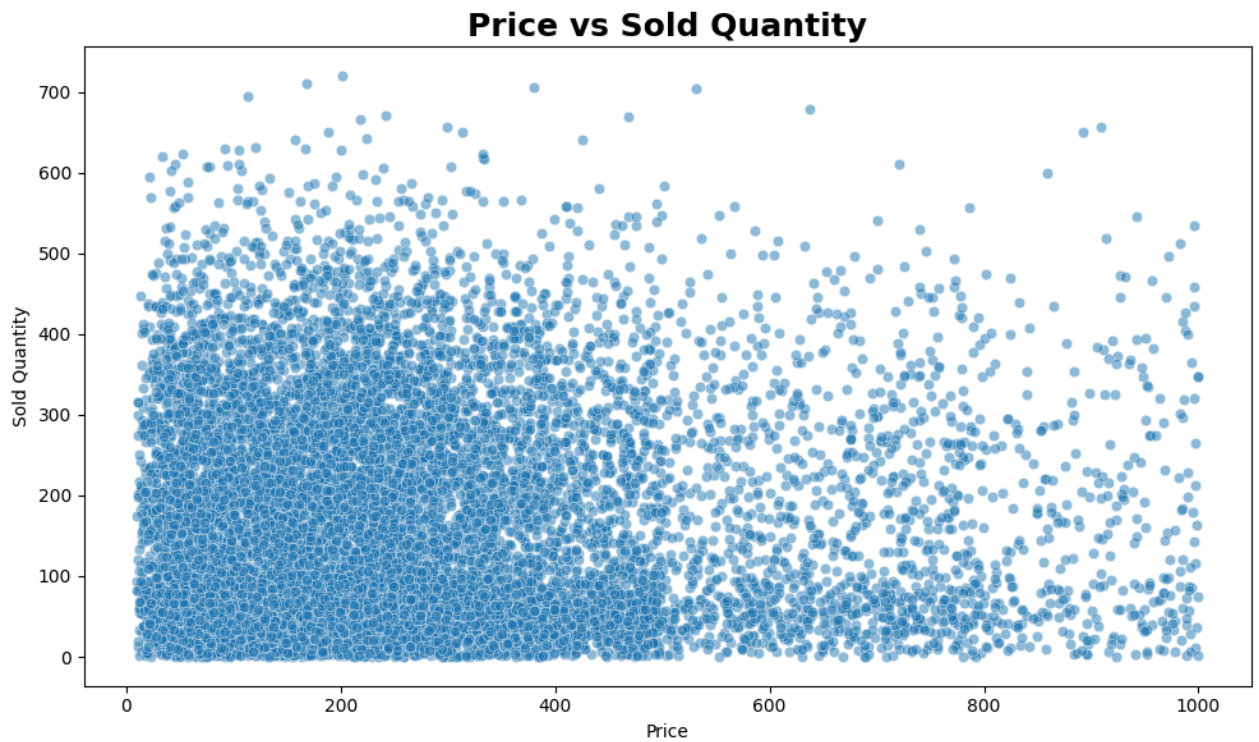
```
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=data,
    x="price",
    y="sold_quantity",
    alpha=0.5
)

plt.title(
    "Price vs Sold Quantity",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel("Price")
plt.ylabel("Sold Quantity")

plt.tight_layout()
plt.show()
```



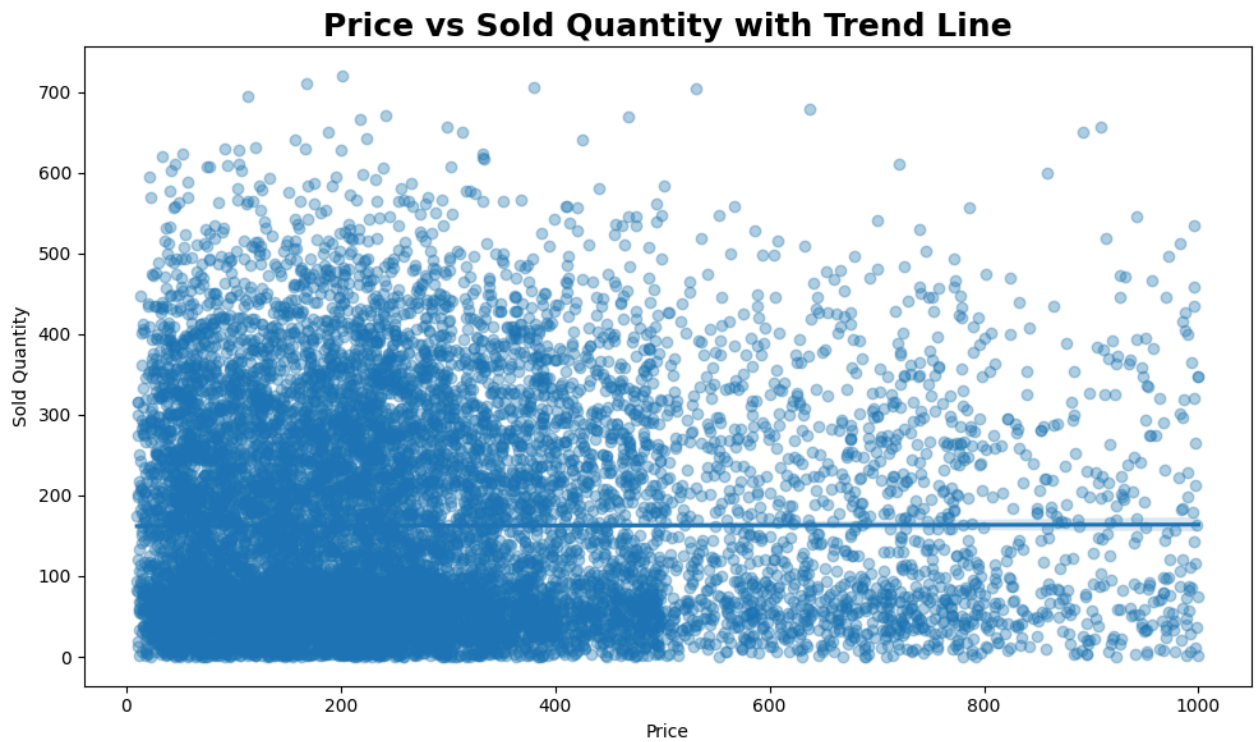
```
plt.figure(figsize=(10, 6))

sns.regplot(
    data=data,
    x="price",
    y="sold_quantity",
    scatter_kws={"alpha": 0.35},
    line_kws={}
)

plt.title(
    "Price vs Sold Quantity with Trend Line",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel("Price")
plt.ylabel("Sold Quantity")

plt.tight_layout()
plt.show()
```



Calculate correlation

```
price_sales_corr = data["price"].corr(data["sold_quantity"])

display("Correlation between Price and Sold Quantity:",
        round(price_sales_corr, 4))
```

```
'Correlation between Price and Sold Quantity:'
np.float64(0.0029)
```

```
fig = px.scatter(
    data,
    x="price",
    y="sold_quantity",
    color="category",
    hover_data=[
        "product_name",
        "brand",
        "rating",
        "discount_pct",
        "demand_index"
    ],
    title="Interactive Price vs Sold Quantity"
)

fig.update_layout(
    template="plotly_white",
    xaxis_title="Price",
    yaxis_title="Sold Quantity"
)
```

```
fig.show()
```

Discount vs Sold Quantity

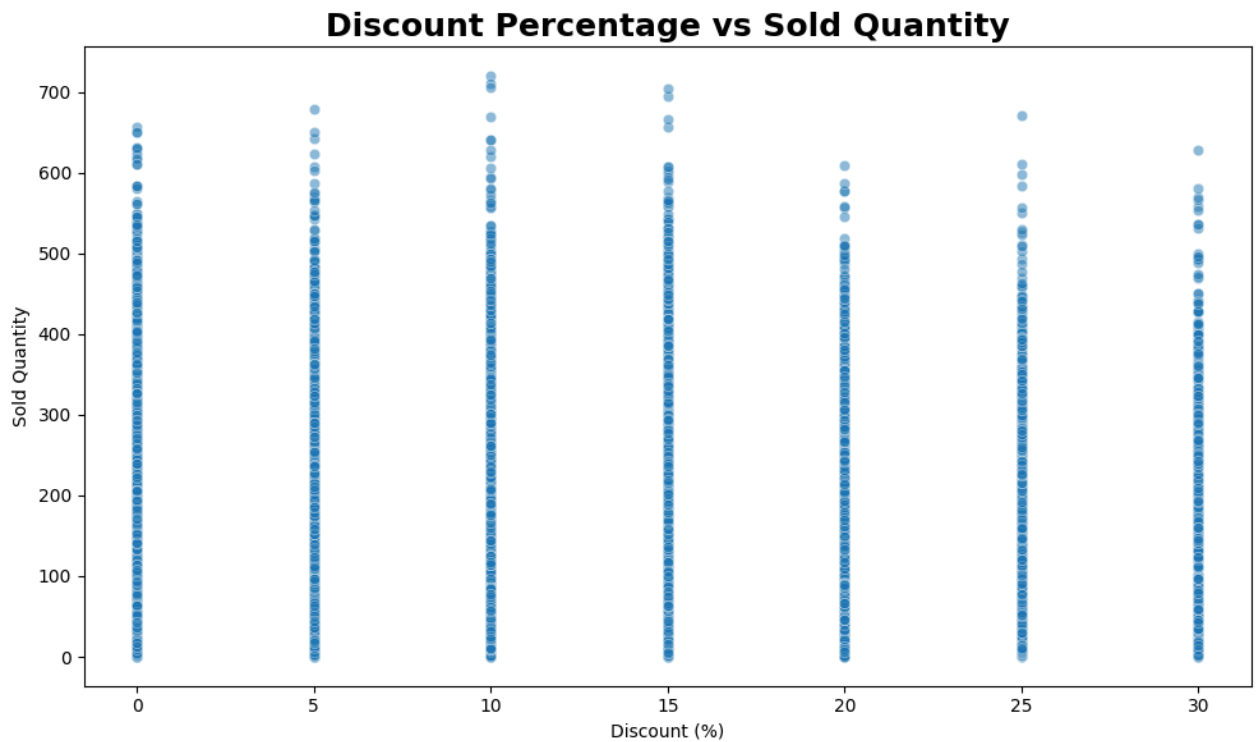
```
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=data,
    x="discount_pct",
    y="sold_quantity",
    alpha=0.5
)

plt.title(
    "Discount Percentage vs Sold Quantity",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel("Discount (%)")
plt.ylabel("Sold Quantity")

plt.tight_layout()
plt.show()
```

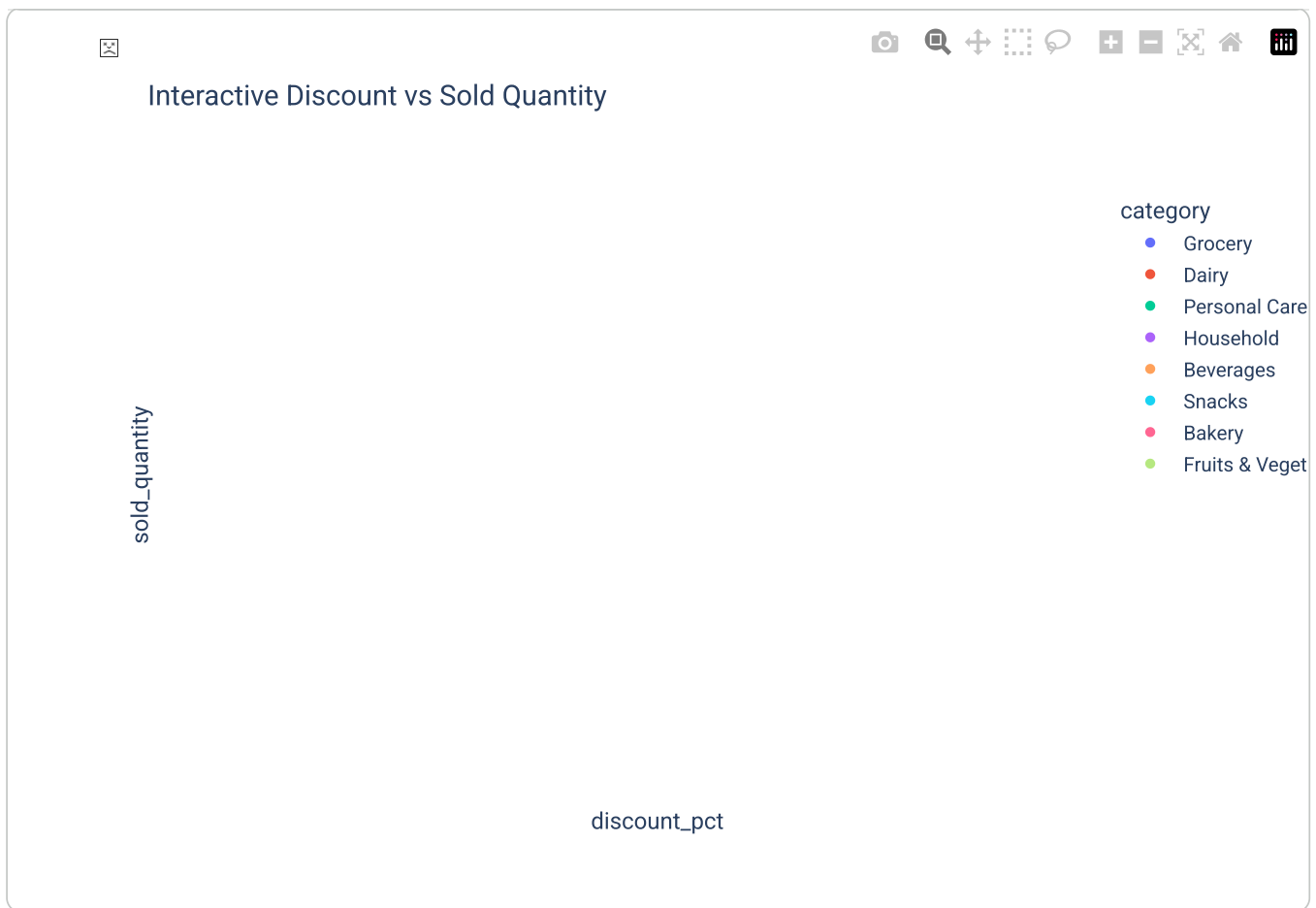


Correlation

```
discount_sales_corr = data["discount_pct"].corr(  
    data["sold_quantity"]  
)  
  
display(  
    "Correlation between Discount and Sold Quantity:",  
    round(discount_sales_corr, 4)  
)
```

```
'Correlation between Discount and Sold Quantity:'  
np.float64(0.0098)
```

```
fig = px.scatter(  
    data,  
    x="discount_pct",  
    y="sold_quantity",  
    color="category",  
    hover_data=["product_name", "brand", "price"],  
    title="Interactive Discount vs Sold Quantity"  
)  
  
fig.update_layout(  
    template="plotly_white"  
)  
  
fig.show()
```



Rating vs Sold Quantity

```
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=data,
    x="rating",
    y="sold_quantity",
    alpha=0.5
)

plt.title(
    "Customer Rating vs Sold Quantity",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel("Rating")
plt.ylabel("Sold Quantity")

plt.tight_layout()
plt.show()
```



Correlation

```
rating_sales_corr = data["rating"].corr(  
    data["sold_quantity"]  
)  
  
display(  
    "Correlation between Rating and Sold Quantity:",  
    round(rating_sales_corr, 4)  
)
```

```
'Correlation between Rating and Sold Quantity:'  
np.float64(-0.0152)
```

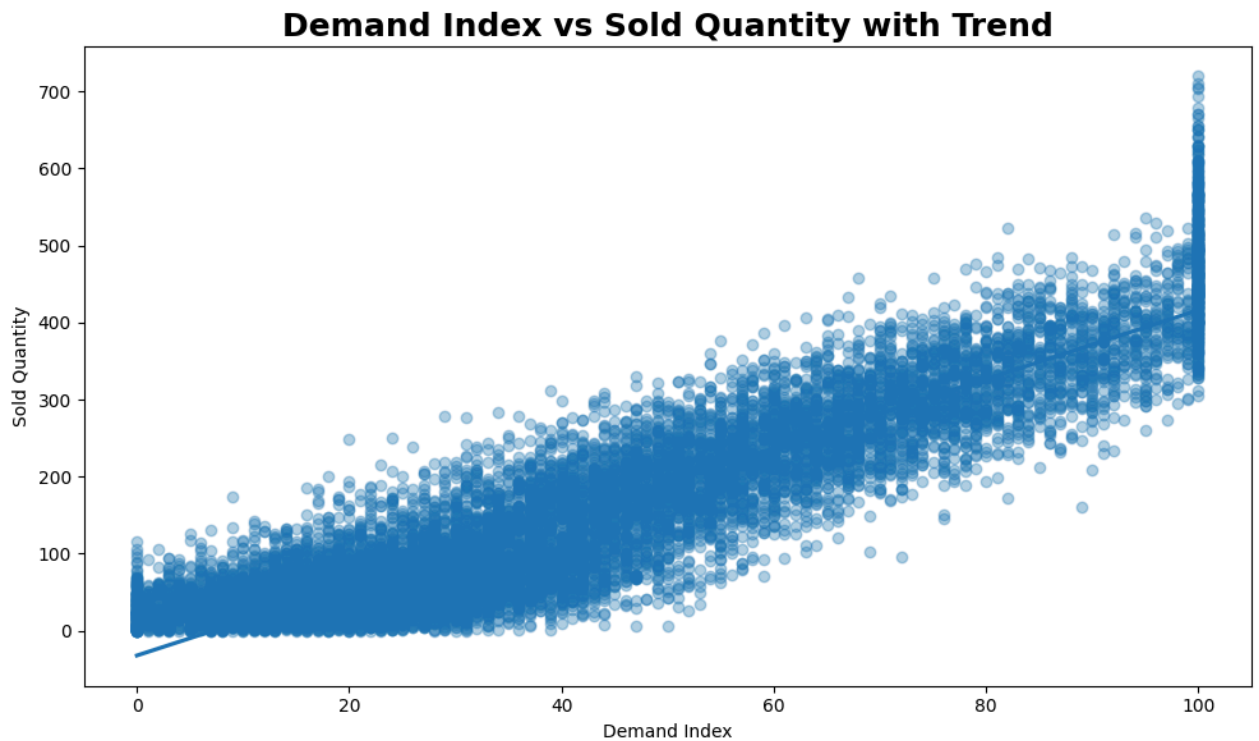
Demand Index vs Sold Quantity

```
plt.figure(figsize=(10, 6))  
  
sns.regplot(  
    data=data,  
    x="demand_index",  
    y="sold_quantity",  
    scatter_kws={"alpha": 0.35},  
    line_kws={}  
)  
  
plt.title(  
    "Demand Index vs Sold Quantity with Trend",  
    fontsize=18,  
    fontweight="bold"
```

```
)

plt.xlabel("Demand Index")
plt.ylabel("Sold Quantity")

plt.tight_layout()
plt.show()
```



Correlation

```
demand_sales_corr = data["demand_index"].corr(
    data["sold_quantity"]
)

display(
    "Correlation between Demand Index and Sold Quantity:",
    round(demand_sales_corr, 4)
)
```

```
'Correlation between Demand Index and Sold Quantity:'
np.float64(0.9143)
```

Stock vs Sold Quantity

```
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=data,
    x="stock",
    y="sold_quantity",
```

```

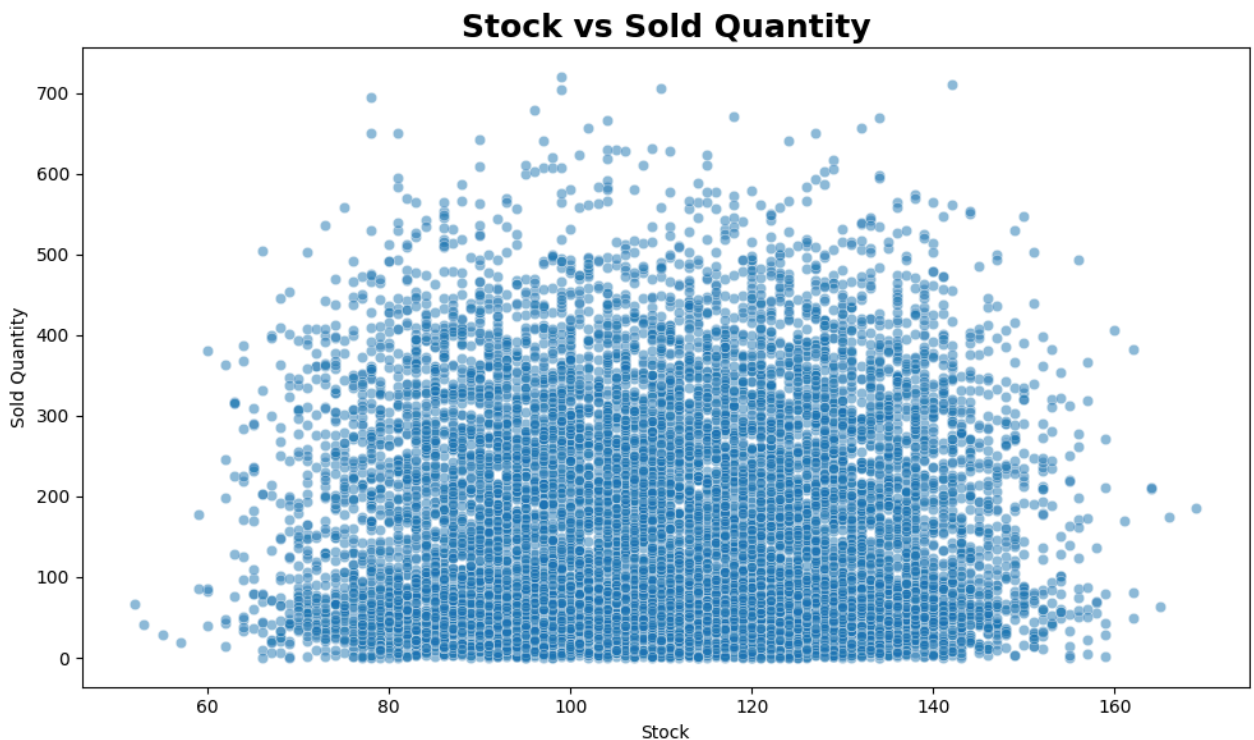
    alpha=0.5
)

plt.title(
    "Stock vs Sold Quantity",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel("Stock")
plt.ylabel("Sold Quantity")

plt.tight_layout()
plt.show()

```



```

plt.figure(figsize=(10, 6))

sns.regplot(
    data=data,
    x="stock",
    y="sold_quantity",
    scatter_kws={"alpha": 0.35},
    line_kws={}
)

plt.title(
    "Stock vs Sold Quantity with Trend",
    fontsize=18,
    fontweight="bold"
)

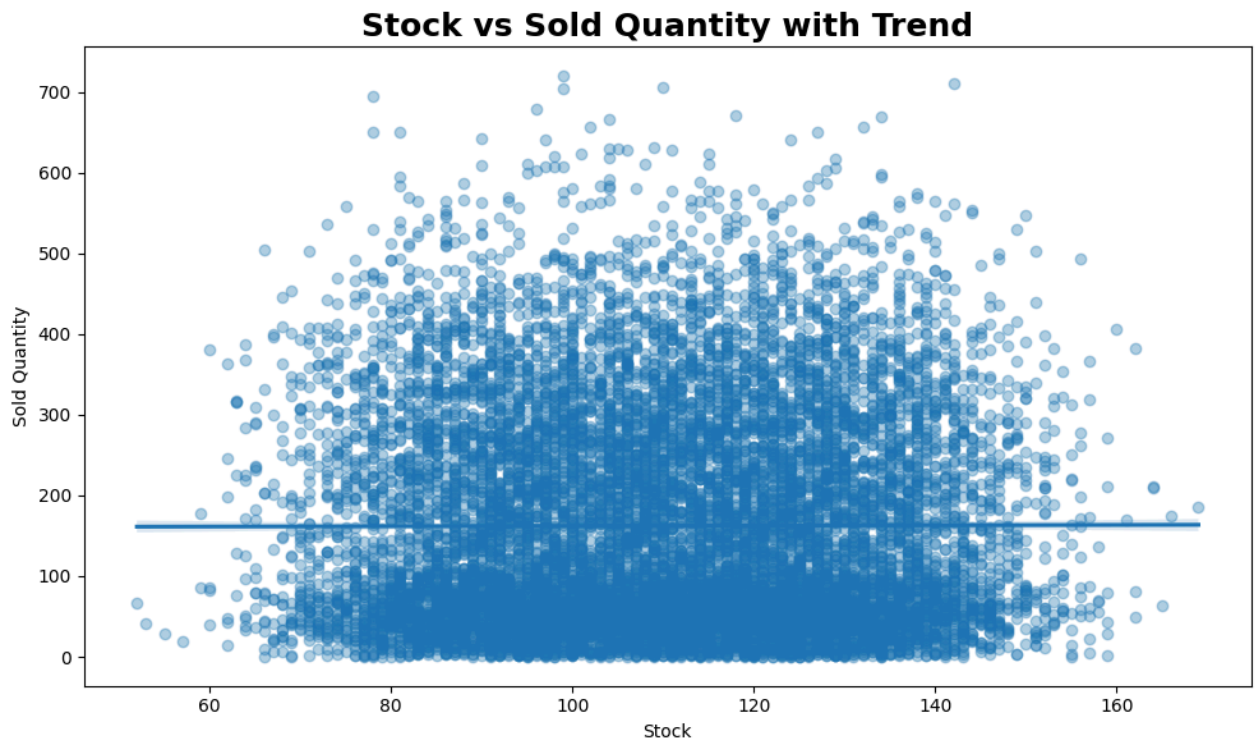
plt.xlabel("Stock")

```

```
plt.ylabel("Sold Quantity")
```

```
plt.tight_layout()
```

```
plt.show()
```



Correlation

```
stock_sales_corr = data["stock"].corr(  
    data["sold_quantity"]  
)  
  
display(  
    "Correlation between Stock and Sold Quantity:",  
    round(stock_sales_corr, 4)  
)
```

```
'Correlation between Stock and Sold Quantity:'  
np.float64(0.003)
```

Delivery Time vs Sold Quantity

```
plt.figure(figsize=(10, 6))  
  
sns.scatterplot(  
    data=data,  
    x="delivery_time_min",  
    y="sold_quantity",  
    alpha=0.5  
)
```

```
plt.title(
    "Delivery Time vs Sold Quantity",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel("Delivery Time (minutes)")
plt.ylabel("Sold Quantity")

plt.tight_layout()
plt.show()
```



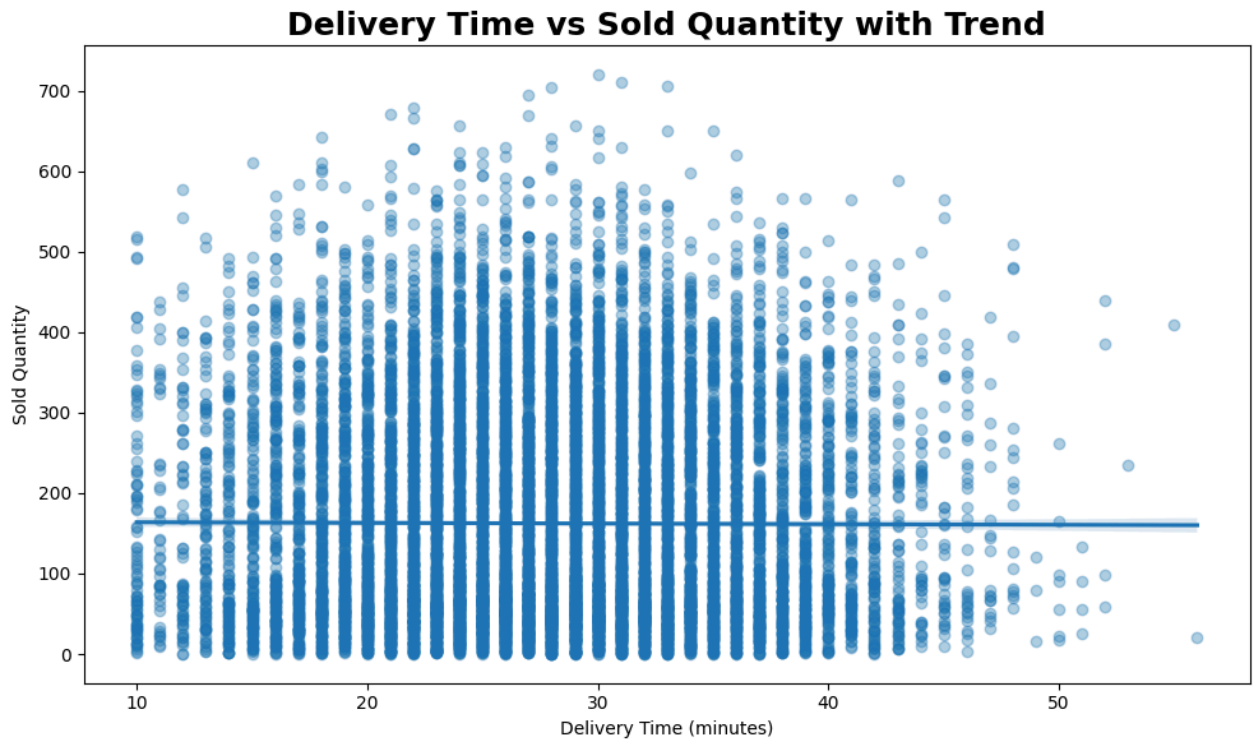
```
plt.figure(figsize=(10, 6))

sns.regplot(
    data=data,
    x="delivery_time_min",
    y="sold_quantity",
    scatter_kws={"alpha": 0.35},
    line_kws={}
)

plt.title(
    "Delivery Time vs Sold Quantity with Trend",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel("Delivery Time (minutes)")
plt.ylabel("Sold Quantity")
```

```
plt.tight_layout()
plt.show()
```



Correlation

```
delivery_sales_corr = data["delivery_time_min"].corr(
    data["sold_quantity"]
)

display(
    "Correlation between Delivery Time and Sold Quantity:",
    round(delivery_sales_corr, 4)
)
```

```
'Correlation between Delivery Time and Sold Quantity:'
np.float64(-0.0043)
```

Complete Correlation Heatmap

```
numeric_data = data.select_dtypes(include=np.number)

correlation_matrix = numeric_data.corr()

plt.figure(figsize=(14, 10))

sns.heatmap(
    correlation_matrix,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
```

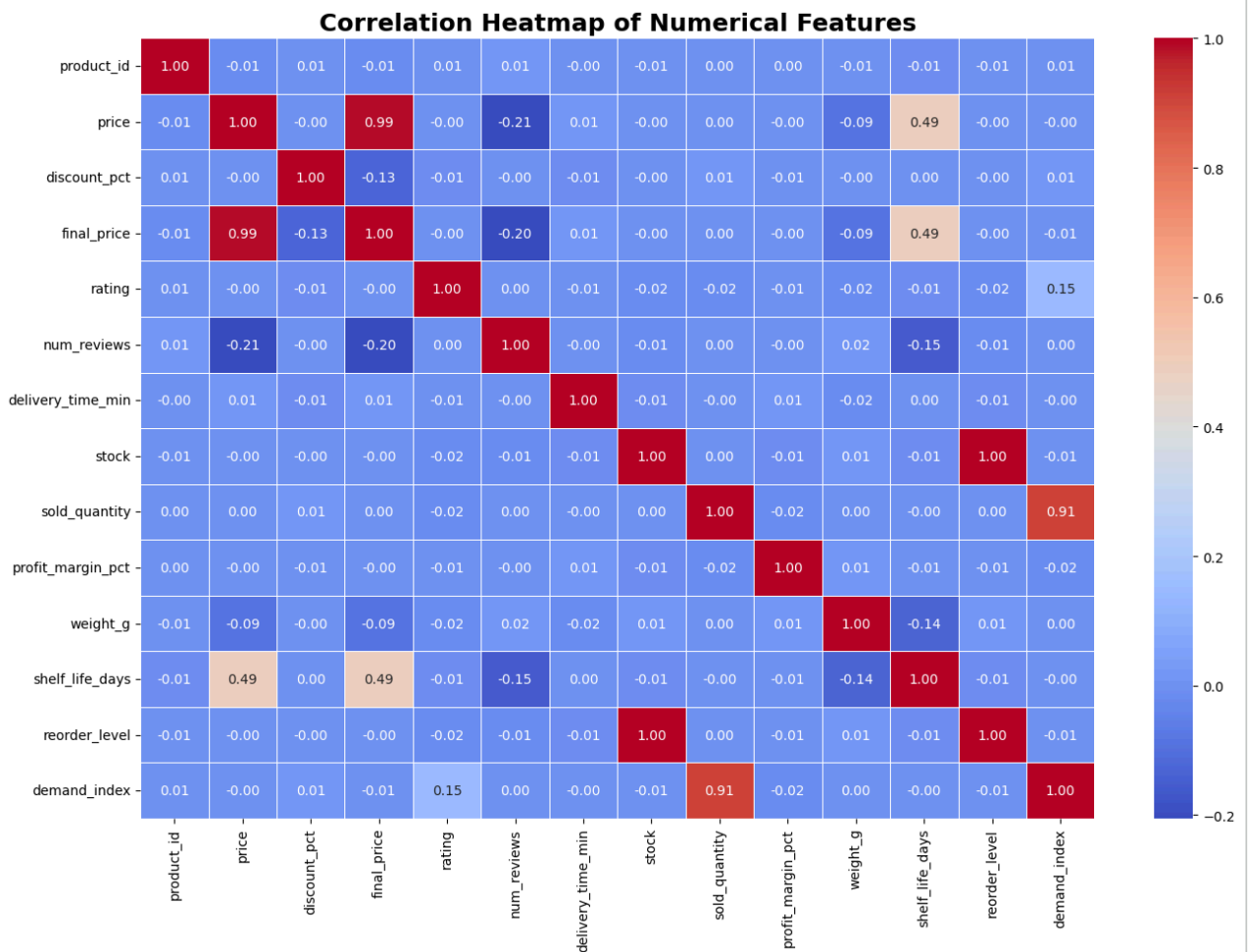
```

        linewidths=0.5
    )

    plt.title(
        "Correlation Heatmap of Numerical Features",
        fontsize=18,
        fontweight="bold"
    )

    plt.tight_layout()
    plt.show()

```



Find correlations specifically with `sold_quantity`

```

target_correlation = (
    correlation_matrix["sold_quantity"]
    .sort_values(ascending=False)
)

display("Correlation with Sold Quantity:")
display(target_correlation)

```

'Correlation with Sold Quantity:'

	sold_quantity
sold_quantity	1.000000
demand_index	0.914314
discount_pct	0.009818
weight_g	0.004152
product_id	0.003915
stock	0.002981
price	0.002942
reorder_level	0.002855
final_price	0.002500
num_reviews	0.000060
shelf_life_days	-0.000135
delivery_time_min	-0.004280
rating	-0.015168
profit_margin_pct	-0.019482

dtype: float64

Sales Trend Over Time

```
data["month"] = data["date_added"].dt.to_period("M").astype(str)
```

```
monthly_sales = (  
    data.groupby("month")["sold_quantity"]  
        .sum()  
        .reset_index()  
)
```

```
display(monthly_sales.head())
```

	month	sold_quantity
0	2023-10	11129
1	2023-11	93295
2	2023-12	91337
3	2024-01	96415
4	2024-02	88498

```
plt.figure(figsize=(14, 6))
```

```
sns.lineplot(  
    data=monthly_sales,
```

```

    x="month",
    y="sold_quantity",
    marker="o"
)

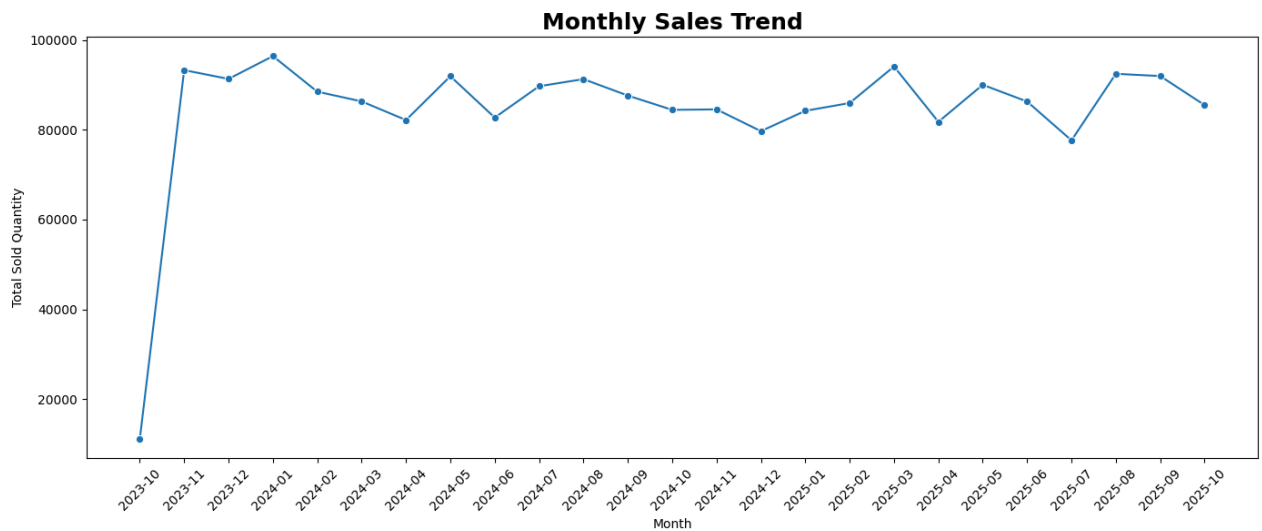
plt.title(
    "Monthly Sales Trend",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel("Month")
plt.ylabel("Total Sold Quantity")

plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

```



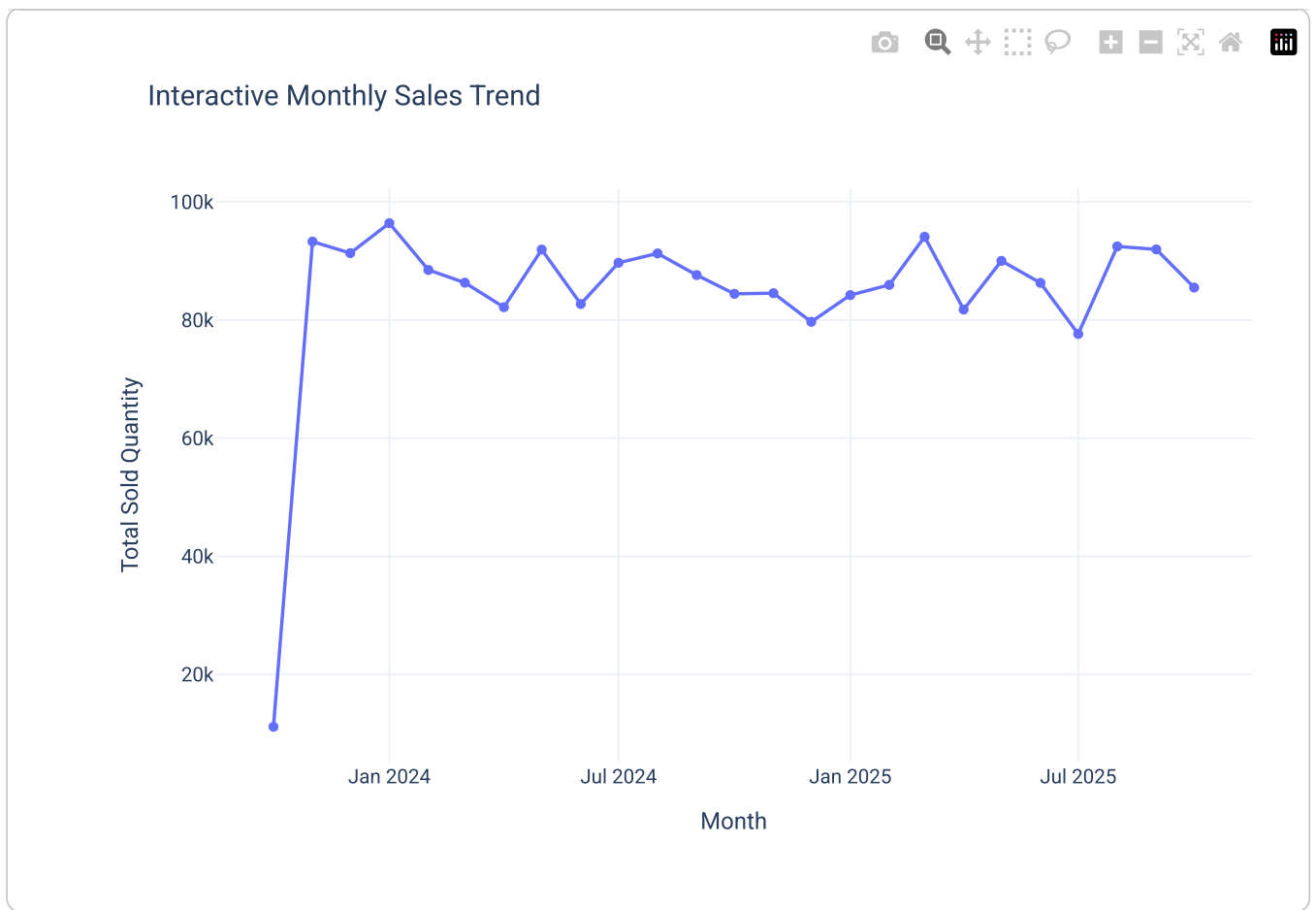
```

fig = px.line(
    monthly_sales,
    x="month",
    y="sold_quantity",
    markers=True,
    title="Interactive Monthly Sales Trend",
    labels={
        "month": "Month",
        "sold_quantity": "Total Sold Quantity"
    }
)

fig.update_layout(
    template="plotly_white"
)

fig.show()

```



✓ Advanced Plotly Analysis

Interactive Sales by Category

```
category_sales_df = (
    data.groupby("category", as_index=False)["sold_quantity"]
    .sum()
    .sort_values("sold_quantity", ascending=False)
)

fig = px.bar(
    category_sales_df,
    x="category",
    y="sold_quantity",
    color="category",
    text="sold_quantity",
    title="Interactive Sales by Category"
)

fig.update_traces(textposition="outside")

fig.update_layout(
    template="plotly_white",
    xaxis_title="Category",
    yaxis_title="Total Sold Quantity",
    showlegend=False
)
```

```
fig.show()
```



Interactive Price vs Sales

```
fig = px.scatter(  
    data,  
    x="price",  
    y="sold_quantity",  
    color="category",  
    size="demand_index",  
    hover_name="product_name",  
    hover_data=[  
        "brand",  
        "rating",  
        "discount_pct",  
        "stock"  
    ],  
    title="Price vs Sold Quantity – Interactive Analysis"  
)  
  
fig.update_layout(  
    template="plotly_white",  
    xaxis_title="Price",  
    yaxis_title="Sold Quantity"  
)  
  
fig.show()
```

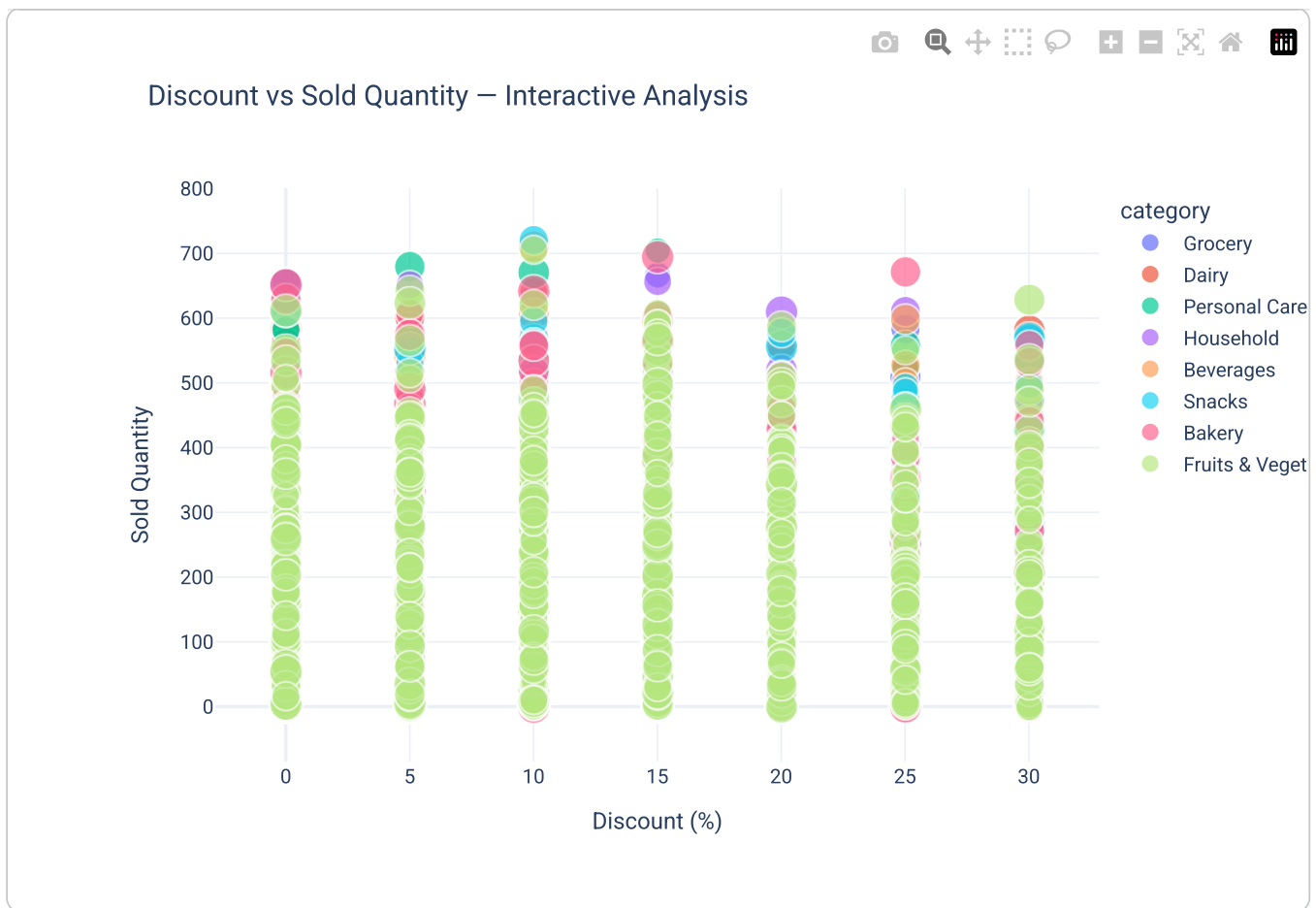


Discount vs Sales

```
fig = px.scatter(
    data,
    x="discount_pct",
    y="sold_quantity",
    color="category",
    size="rating",
    hover_name="product_name",
    hover_data=[
        "price",
        "final_price",
        "brand",
        "demand_index"
    ],
    title="Discount vs Sold Quantity – Interactive Analysis"
)

fig.update_layout(
    template="plotly_white",
    xaxis_title="Discount (%)",
    yaxis_title="Sold Quantity"
)

fig.show()
```

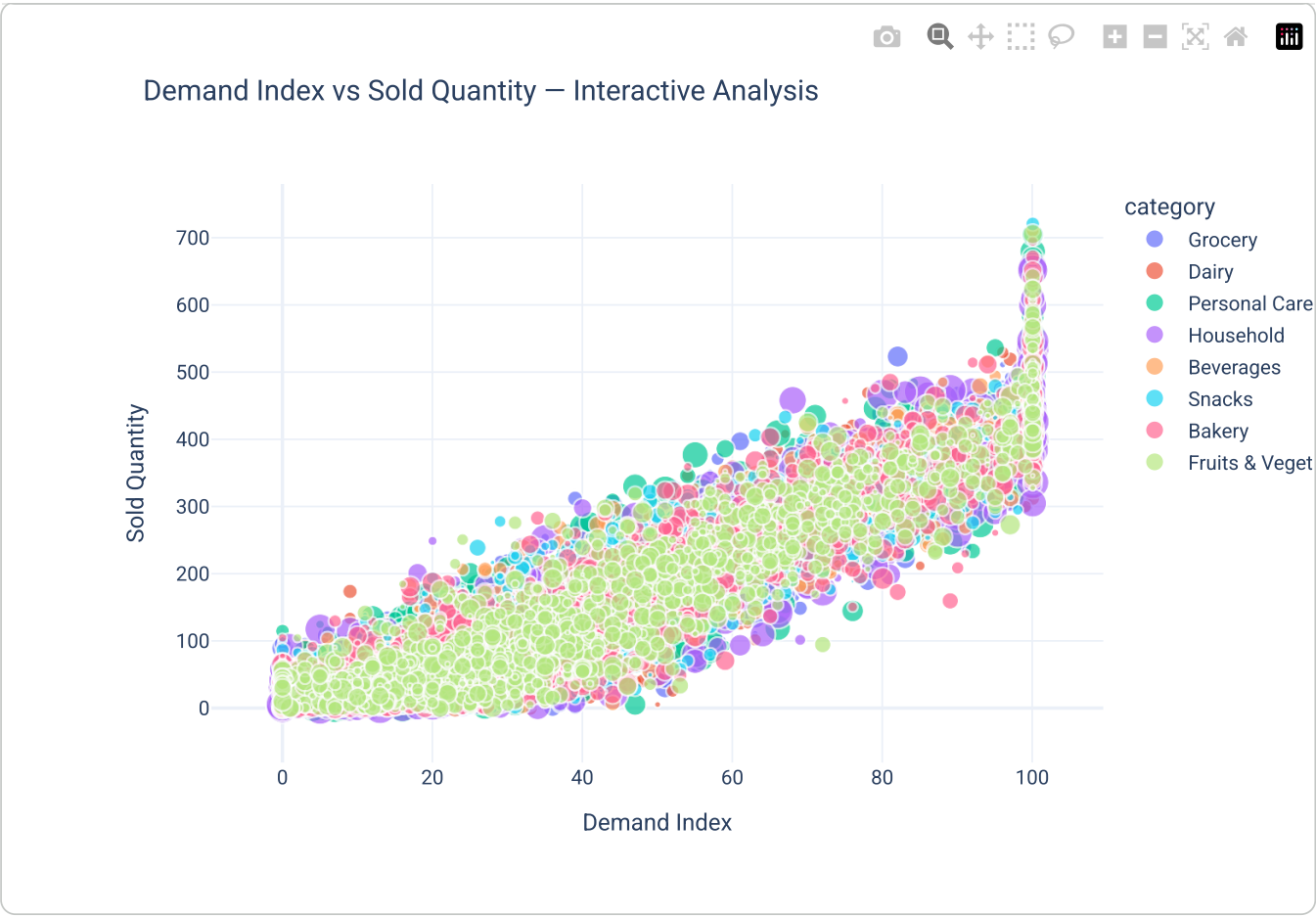


Demand vs Sales

```
fig = px.scatter(
    data,
    x="demand_index",
    y="sold_quantity",
    color="category",
    size="final_price",
    hover_name="product_name",
    hover_data=[
        "brand",
        "rating",
        "discount_pct",
        "stock"
    ],
    title="Demand Index vs Sold Quantity – Interactive Analysis"
)

fig.update_layout(
    template="plotly_white",
    xaxis_title="Demand Index",
    yaxis_title="Sold Quantity"
)

fig.show()
```



Price × Rating × Sold Quantity



```

fig = px.scatter_3d(
    data,
    x="price",
    y="rating",
    z="sold_quantity",
    color="category",
    size="demand_index",
    hover_name="product_name",
    hover_data=[
        "brand",
        "discount_pct",
        "stock"
    ],
    title="Price, Rating and Sold Quantity"
)

fig.update_layout(
    template="plotly_white",
    scene=dict(
        xaxis_title="Price",
        yaxis_title="Rating",
        zaxis_title="Sold Quantity"
    )
)

fig.show()

```



Price, Rating and Sold Quantity



category

- Grocery
- Dairy
- Personal Care
- Household
- Beverages
- Snacks
- Bakery
- Fruits & Veget

Discount × Demand Index × Sold Quantity

```

fig = px.scatter_3d(
    data,
    x="discount_pct",
    y="demand_index",
    z="sold_quantity",
    color="category",
    size="rating",
    hover_name="product_name",
    title="Discount, Demand and Sales"
)

fig.update_layout(
    template="plotly_white",
    scene=dict(
        xaxis_title="Discount (%)",
        yaxis_title="Demand Index",
        zaxis_title="Sold Quantity"
    )
)

fig.show()

```



Discount, Demand and Sales



category

- Grocery
- Dairy
- Personal Care
- Household
- Beverages
- Snacks
- Bakery
- Fruits & Veget

✓ Feature Engineering

ML dataset

```
model_df = data.copy()

print("Initial Shape:", model_df.shape)
```

Initial Shape: (13000, 26)

```
model_df["year"] = model_df["date_added"].dt.year
model_df["month"] = model_df["date_added"].dt.month
model_df["day"] = model_df["date_added"].dt.day
model_df["day_of_week"] = model_df["date_added"].dt.dayofweek
```

```
display(
    model_df[
        ["date_added", "year", "month", "day", "day_of_week"]
    ].head()
)
```

	date_added	year	month	day	day_of_week
0	2023-11-27	2023	11	27	0
1	2024-08-07	2024	8	7	2
2	2024-03-03	2024	3	3	6
3	2024-08-07	2024	8	7	2
4	2024-07-04	2024	7	4	3

Create weekend feature

```
model_df["is_weekend"] = (
    model_df["day_of_week"] >= 5
).astype(int)
```

Calculate discount amount

```
model_df["discount_amount"] = (
    model_df["price"] * model_df["discount_pct"] / 100
)
```

Calculate price after discount

```
model_df["calculated_final_price"] = (
    model_df["price"] - model_df["discount_amount"]
)
```

```
display(
    model_df[
        [
            "price",
            "discount_pct",
            "discount_amount",
```

```

        "final_price",
        "calculated_final_price"
    ]
    ].head()
)

```

	price	discount_pct	discount_amount	final_price	calculated_final_price
0	199.78	25	49.9450	149.84	149.8350
1	44.32	30	13.2960	31.02	31.0240
2	501.13	0	0.0000	501.13	501.1300
3	627.17	0	0.0000	627.17	627.1700
4	101.69	15	15.2535	86.44	86.4365

Create inventory utilization

```

model_df["inventory_utilization"] = (
    model_df["sold_quantity"] /
    (model_df["stock"] + model_df["sold_quantity"])
)

```

Convert to percentage:

```

model_df["inventory_utilization_pct"] = (
    model_df["inventory_utilization"] * 100
)

```

Create review engagement feature

```

model_df["review_engagement"] = (
    model_df["num_reviews"] /
    (model_df["sold_quantity"] + 1)
)

```

Create product age

```

reference_date = model_df["date_added"].max()

model_df["product_age_days"] = (
    reference_date - model_df["date_added"]
).dt.days

```

Create expiry duration

```

model_df["days_until_expiry"] = (
    model_df["expiry_date"] -
    model_df["date_added"]
).dt.days

```

Create price-per-gram

```
model_df["price_per_gram"] = (  
    model_df["final_price"] /  
    model_df["weight_g"].replace(0, np.nan)  
)
```

Create price-per-100g

```
model_df["price_per_100g"] = (  
    model_df["final_price"] /  
    model_df["weight_g"].replace(0, np.nan)  
) * 100
```

Check the engineered features

```
new_features = [  
    "year",  
    "month",  
    "day",  
    "day_of_week",  
    "is_weekend",  
    "discount_amount",  
    "calculated_final_price",  
    "product_age_days",  
    "days_until_expiry",  
    "price_per_gram",  
    "price_per_100g"  
]  
  
print("New Features:")  
display(model_df[new_features].head())
```

New Features:

	year	month	day	day_of_week	is_weekend	discount_amount	calculated_final_price	price_per_gram	price_per_100g
0	2023	11	27	0	0	49.9450	149.8350	3.0000	300.0000
1	2024	8	7	2	0	13.2960	31.0240	3.0000	300.0000
2	2024	3	3	6	1	0.0000	501.1300	3.0000	300.0000
3	2024	8	7	2	0	0.0000	627.1700	3.0000	300.0000
4	2024	7	4	3	0	15.2535	86.4365	3.0000	300.0000

Check missing values created by engineering

```
print("Missing Values After Feature Engineering:")
display(
    model_df[new_features].isnull().sum()
)
```

Missing Values After Feature Engineering:

	0
year	0
month	0
day	0
day_of_week	0
is_weekend	0
discount_amount	0
calculated_final_price	0
product_age_days	0
days_until_expiry	0
price_per_gram	0
price_per_100g	0

dtype: int64

Check infinite values

```
display(
    "Infinite Values:",
    np.isinf(
        model_df.select_dtypes(include=np.number)
    ).sum().sum()
)
```

'Infinite Values:'
np.int64(0)

```
model_df.replace(
    [np.inf, -np.inf],
    np.nan,
    inplace=True
)
```

Final feature-engineered dataset

```
display("Feature-Engineered Dataset Shape:")
display(model_df.shape)

display("\nColumns:")
display(model_df.columns.tolist())
```

```
'Feature-Engineered Dataset Shape:'  
(13000, 39)  
'\nColumns:'  
['product_id',  
 'product_name',  
 'category',  
 'brand',  
 'price',  
 'discount_pct',  
 'final_price',  
 'rating',  
 'num_reviews',  
 'delivery_time_min',  
 'city',  
 'seller',  
 'stock',  
 'sold_quantity',  
 'profit_margin_pct',  
 'is_organic',  
 'packaging_type',  
 'weight_g',  
 'shelf_life_days',  
 'reorder_level',  
 'demand_index',  
 'date_added',  
 'expiry_date',  
 'offer_type',  
 'delivery_status',  
 'month',  
 'year',  
 'day',  
 'day_of_week',  
 'is_weekend',  
 'discount_amount',  
 'calculated_final_price',  
 'inventory_utilization',  
 'inventory_utilization_pct',  
 'review_engagement',  
 'product_age_days',  
 'days_until_expiry',  
 'price_per_gram',  
 'price_per_100g']
```

✓ ML Preparation

Define Target & Features

```
target = "sold_quantity"  
  
features = [  
    "price",  
    "final_price",  
    "discount_pct",  
    "rating",  
    "num_reviews",  
    "delivery_time_min",  
    "stock",  
    "profit_margin_pct",  
    "weight_g",
```

```

        "shelf_life_days",
        "demand_index",
        "is_organic",
        "year",
        "month",
        "day_of_week",
        "is_weekend",
        "discount_amount",
        "product_age_days",
        "days_until_expiry",
        "price_per_gram",
        "price_per_100g"
    ]

X = model_df[features].copy()
y = model_df[target].copy()

print("ML DATA PREPARATION")

print("\nX Shape:", X.shape)
print("y Shape:", y.shape)

print("\nTarget Variable:")
print(target)

print("\nFeature Count:")
print(len(features))

print("\nFeatures:")
print(features)

```

ML DATA PREPARATION

X Shape: (13000, 21)
y Shape: (13000,)

Target Variable:
sold_quantity

Feature Count:
21

Features:
['price', 'final_price', 'discount_pct', 'rating', 'num_reviews', 'delivery_time_min', '

Check Data Types

```

display("\n DATA TYPES")
display(X.dtypes)

```

'\n DATA TYPES '

0

price	float64
final_price	float64
discount_pct	int64
rating	float64
num_reviews	int64
delivery_time_min	int64
stock	int64
profit_margin_pct	float64
weight_g	int64
shelf_life_days	int64
demand_index	int64
is_organic	bool
year	int32
month	int32
day_of_week	int32
is_weekend	int64
discount_amount	float64
product_age_days	int64
days_until_expiry	int64
price_per_gram	float64
price_per_100g	float64

dtype: object

Separate Numerical & Categorical Features

```
numeric_features = X.select_dtypes(
    include=["int64", "float64", "int32", "float32", "bool"]
).columns.tolist()

categorical_features = X.select_dtypes(
    include=["object", "category"]
).columns.tolist()

print("\nNUMERICAL FEATURES")
print(numeric_features)

print("\nNumber of Numerical Features:", len(numeric_features))

print("\nCATEGORICAL FEATURES")
print(categorical_features)
```

```
print("\nNumber of Categorical Features:", len(categorical_features))
```

NUMERICAL FEATURES

```
['price', 'final_price', 'discount_pct', 'rating', 'num_reviews', 'delivery_time_min', '
```

Number of Numerical Features: 21

CATEGORICAL FEATURES

```
[]
```

Number of Categorical Features: 0

Check Missing Values

```
print("\n MISSING VALUES")

missing_values = X.isnull().sum()

print(missing_values[missing_values > 0])

print("\nTotal Missing Values:", X.isnull().sum().sum())
print("Target Missing Values:", y.isnull().sum())
```

```
MISSING VALUES
Series([], dtype: int64)

Total Missing Values: 0
Target Missing Values: 0
```

Create Preprocessing Pipeline

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# Numerical preprocessing
numeric_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]
)

# Categorical preprocessing
categorical_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore"))
    ]
)

# Combine preprocessing
```

```

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)

print("Preprocessing Pipeline Created Successfully!")

```

Preprocessing Pipeline Created Successfully!

Final ML Dataset Check

```

print("FINAL ML DATA CHECK")

print("X Shape:", X.shape)
print("y Shape:", y.shape)

print("\nX Missing Values:", X.isnull().sum().sum())
print("y Missing Values:", y.isnull().sum())

print("\nTarget Statistics:")
display(y.describe())

```

FINAL ML DATA CHECK
X Shape: (13000, 21)
y Shape: (13000,)

X Missing Values: 0
y Missing Values: 0

Target Statistics:

	sold_quantity
count	13000.000000
mean	162.400077
std	132.742425
min	0.000000
25%	54.000000
50%	120.000000
75%	253.000000
max	720.000000

dtype: float64

Important Leakage Check

```

leakage_features = [
    "inventory_utilization",
    "inventory_utilization_pct",
    "review_engagement"
]

```

```
print("\nExcluded Leakage Features:")
display(leakage_features)
```

```
Excluded Leakage Features:
['inventory_utilization', 'inventory_utilization_pct', 'review_engagement']
```

Train/Test Split

```
print("Model DataFrame Columns:")
print(model_df.columns.tolist())
```

```
Model DataFrame Columns:
['product_id', 'product_name', 'category', 'brand', 'price', 'discount_pct', 'final_price', 'rating', 'num_reviews']
```

```
print("Model DataFrame Shape:", model_df.shape)

display(model_df.head())
```

```
Model DataFrame Shape: (13000, 39)
```

	product_id	product_name	category	brand	price	discount_pct	final_price	rating	num_reviews
0	1	Tata Organic Grocery 300	Grocery	Tata	199.78	25	149.84	4.5	
1	2	Mother Dairy Lite Dairy 275	Dairy	Mother Dairy	44.32	30	31.02	4.0	
2	3	P&G Classic Personal 439	Personal Care	P&G	501.13	0	501.13	3.7	
3	4	Dettol Fresh Household 771	Household	Dettol	627.17	0	627.17	3.9	
4	5	Minute Maid Daily Beverages 264	Beverages	Minute Maid	101.69	15	86.44	4.3	

```
5 rows × 39 columns
```

```
print("model_df Shape:", model_df.shape)

print("\nmodel_df Columns:")
for i, col in enumerate(model_df.columns, 1):
    print(i, "→", col)
```

```
model_df Shape: (13000, 39)
```

```
model_df Columns:
1 → product_id
2 → product_name
3 → category
4 → brand
5 → price
6 → discount_pct
7 → final_price
8 → rating
```

```

9 → num_reviews
10 → delivery_time_min
11 → city
12 → seller
13 → stock
14 → sold_quantity
15 → profit_margin_pct
16 → is_organic
17 → packaging_type
18 → weight_g
19 → shelf_life_days
20 → reorder_level
21 → demand_index
22 → date_added
23 → expiry_date
24 → offer_type
25 → delivery_status
26 → month
27 → year
28 → day
29 → day_of_week
30 → is_weekend
31 → discount_amount
32 → calculated_final_price
33 → inventory_utilization
34 → inventory_utilization_pct
35 → review_engagement
36 → product_age_days
37 → days_until_expiry
38 → price_per_gram
39 → price_per_100g

```

```

print("\nFirst 5 Rows:")
display(model_df.head())

```

First 5 Rows:

	product_id	product_name	category	brand	price	discount_pct	final_price	rating	num_reviews
0	1	Tata Organic Grocery 300	Grocery	Tata	199.78	25	149.84	4.5	
1	2	Mother Dairy Lite Dairy 275	Dairy	Mother Dairy	44.32	30	31.02	4.0	
2	3	P&G Classic Personal 439	Personal Care	P&G	501.13	0	501.13	3.7	
3	4	Dettol Fresh Household 771	Household	Dettol	627.17	0	627.17	3.9	
4	5	Minute Maid Daily Beverages 264	Beverages	Minute Maid	101.69	15	86.44	4.3	

5 rows × 39 columns

```
target = "Item_Outlet_Sales"
```

```
print(model_df.columns.tolist())
```

```
['product_id', 'product_name', 'category', 'brand', 'price', 'discount_pct', 'final_price']
```

```
# Display available columns
print("Available Columns:")
print(model_df.columns.tolist())
```

```
Available Columns:
['product_id', 'product_name', 'category', 'brand', 'price', 'discount_pct', 'final_price']
```

```
# Display numeric columns
numeric_columns = model_df.select_dtypes(include="number").columns.tolist()

print("\nNumeric Columns:")
print(numeric_columns)
```

```
Numeric Columns:
['product_id', 'price', 'discount_pct', 'final_price', 'rating', 'num_reviews', 'delivery_time']
```

```
# Select target
target = numeric_columns[-1]

print("Selected Target:", target)

# Features and target
X = model_df.drop(columns=[target])
y = model_df[target]

print("\nX Shape:", X.shape)
print("y Shape:", y.shape)
```

```
Selected Target: price_per_100g
```

```
X Shape: (13000, 38)
y Shape: (13000,)
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("TRAIN / TEST SPLIT")

print("X_train Shape:", X_train.shape)
print("X_test Shape :", X_test.shape)
print("y_train Shape:", y_train.shape)
print("y_test Shape :", y_test.shape)
```

```
TRAIN / TEST SPLIT
X_train Shape: (10400, 38)
X_test Shape : (2600, 38)
```

```
y_train Shape: (10400,)
y_test Shape : (2600,)
```

```
print("\nTraining Samples:", len(X_train))
print("Testing Samples :", len(X_test))

print("\nTraining Percentage:",
      round(len(X_train) / len(X) * 100, 2), "%")

display("Testing Percentage:",
      round(len(X_test) / len(X) * 100, 2), "%")
```

```
Training Samples: 10400
Testing Samples : 2600

Training Percentage: 80.0 %
'Testing Percentage:'
20.0
'%'
```

```
import pandas as pd

split_summary = pd.DataFrame({
    "Dataset": ["Training", "Testing"],
    "Samples": [len(X_train), len(X_test)],
    "Percentage": [
        round(len(X_train) / len(X) * 100, 2),
        round(len(X_test) / len(X) * 100, 2)
    ]
})

display(split_summary)
```

	Dataset	Samples	Percentage
0	Training	10400	80.0
1	Testing	2600	20.0

Model Training

```
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
import pandas as pd

# Copy training and testing data
X_train = X_train.copy()
X_test = X_test.copy()

# Convert categorical columns into numerical values
categorical_cols = X_train.select_dtypes(include=["object", "category"]).columns

for col in categorical_cols:
    encoder = LabelEncoder()
```

```

# Fit on combined train + test values
combined_values = pd.concat([X_train[col], X_test[col]]).astype(str)
encoder.fit(combined_values)

X_train[col] = encoder.transform(X_train[col].astype(str))
X_test[col] = encoder.transform(X_test[col].astype(str))

# Drop any datetime columns that cannot be directly handled by sklearn
datetime_cols = X_train.select_dtypes(include=["datetime64", "datetime64[ns]").columns
X_train = X_train.drop(columns=datetime_cols, errors='ignore')
X_test = X_test.drop(columns=datetime_cols, errors='ignore')

# Train Linear Regression
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

# Train Random Forest Regression
rf_model = RandomForestRegressor(
    n_estimators=200,
    random_state=42
)
rf_model.fit(X_train, y_train)

# Display training status
print("MODEL TRAINING COMPLETED")

display("Linear Regression: Trained Successfully")
display("Random Forest Regression: Trained Successfully")

display("Number of Training Samples:", len(X_train))
display("Number of Features:", X_train.shape[1])

```

```

MODEL TRAINING COMPLETED
'Linear Regression: Trained Successfully'
'Random Forest Regression: Trained Successfully'
'Number of Training Samples:'
10400
'Number of Features:'
36

```

Model Evaluation

```

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

# MODEL PREDICTIONS

linear_pred = linear_model.predict(X_test)
rf_pred = rf_model.predict(X_test)

# LINEAR REGRESSION

```

```

linear_mae = mean_absolute_error(y_test, linear_pred)
linear_mse = mean_squared_error(y_test, linear_pred)
linear_rmse = np.sqrt(linear_mse)
linear_r2 = r2_score(y_test, linear_pred)

# RANDOM FOREST METRICS

rf_mae = mean_absolute_error(y_test, rf_pred)
rf_mse = mean_squared_error(y_test, rf_pred)
rf_rmse = np.sqrt(rf_mse)
rf_r2 = r2_score(y_test, rf_pred)

print("MODEL EVALUATION METRICS")

display(
    pd.DataFrame({
        "Model": [
            "Linear Regression",
            "Random Forest Regression"
        ],
        "MAE": [
            round(linear_mae, 4),
            round(rf_mae, 4)
        ],
        "MSE": [
            round(linear_mse, 4),
            round(rf_mse, 4)
        ],
        "RMSE": [
            round(linear_rmse, 4),
            round(rf_rmse, 4)
        ],
        "R2 Score": [
            round(linear_r2, 4),
            round(rf_r2, 4)
        ]
    })
)

```

MODEL EVALUATION METRICS

	Model	MAE	MSE	RMSE	R ² Score
0	Linear Regression	0.0000	0.0000	0.0000	1.0000
1	Random Forest Regression	0.0936	1.5364	1.2395	0.9999

Actual vs Predicted — Random Forest

```

plt.figure(figsize=(10, 6))

sns.scatterplot(
    x=y_test,
    y=rf_pred,
    alpha=0.6
)

```

```

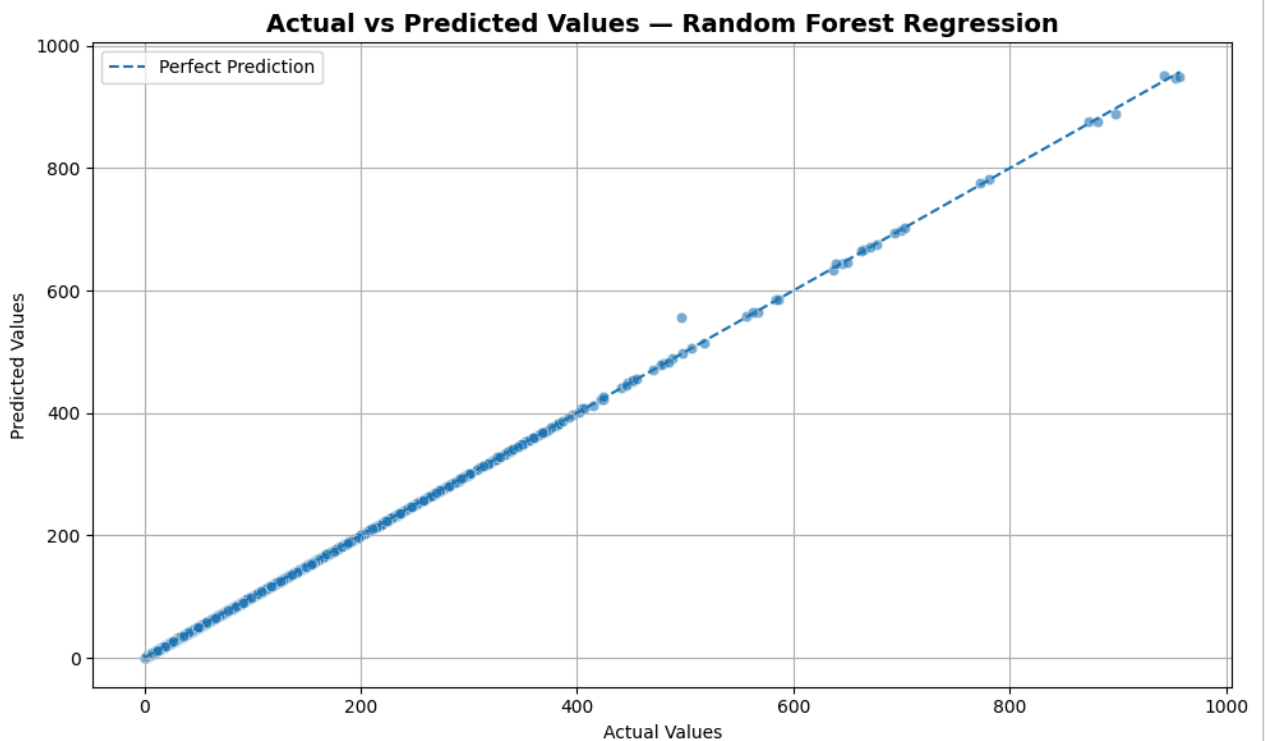
min_val = min(y_test.min(), rf_pred.min())
max_val = max(y_test.max(), rf_pred.max())

plt.plot(
    [min_val, max_val],
    [min_val, max_val],
    linestyle="--",
    label="Perfect Prediction"
)

plt.title(
    "Actual vs Predicted Values – Random Forest Regression",
    fontsize=14,
    fontweight="bold"
)

plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```



Prediction

```

# Use Random Forest as the final model
final_model = rf_model

# Generate predictions
y_pred = final_model.predict(X_test)

```

```
# Create prediction comparison table
prediction_results = pd.DataFrame({
    "Actual Value": y_test.values,
    "Predicted Value": y_pred
})

# Calculate prediction error
prediction_results["Error"] = (
    prediction_results["Actual Value"]
    - prediction_results["Predicted Value"]
)

# Display prediction results
print("          MODEL PREDICTION ")

display(prediction_results.head(20))
```

	MODEL PREDICTION		
	Actual Value	Predicted Value	Error
0	59.980000	59.949040	0.030960
1	74.602000	74.595658	0.006342
2	636.370000	634.397650	1.972350
3	26.072000	26.071436	0.000564
4	151.000000	151.025663	-0.025663
5	248.848000	248.628313	0.219687
6	16.512667	16.512956	-0.000289
7	19.989000	19.987543	0.001457
8	55.112000	55.078867	0.033133
9	28.613333	28.624097	-0.010763
10	33.213000	33.183895	0.029105
11	54.548000	54.522299	0.025701
12	15.879000	15.877095	0.001905
13	45.480000	45.529207	-0.049207
14	45.270000	45.297424	-0.027424
15	154.670000	154.702970	-0.032970
16	26.546667	26.554357	-0.007690
17	2.244000	2.235393	0.008607
18	240.170000	240.071608	0.098392
19	247.336000	247.339514	-0.003514

Prediction Summary

```
print("PREDICTION SUMMARY")
```

```
print("Total Predictions:", len(y_pred))
print("Average Actual Value:", round(y_test.mean(), 4))
print("Average Predicted Value:", round(y_pred.mean(), 4))
print("Average Prediction Error:",
      round(prediction_results["Error"].mean(), 4))
```

PREDICTION SUMMARY
Total Predictions: 2600
Average Actual Value: 81.7289
Average Predicted Value: 81.7508
Average Prediction Error: -0.0219

Prediction Visualization

```
plt.figure(figsize=(12, 6))

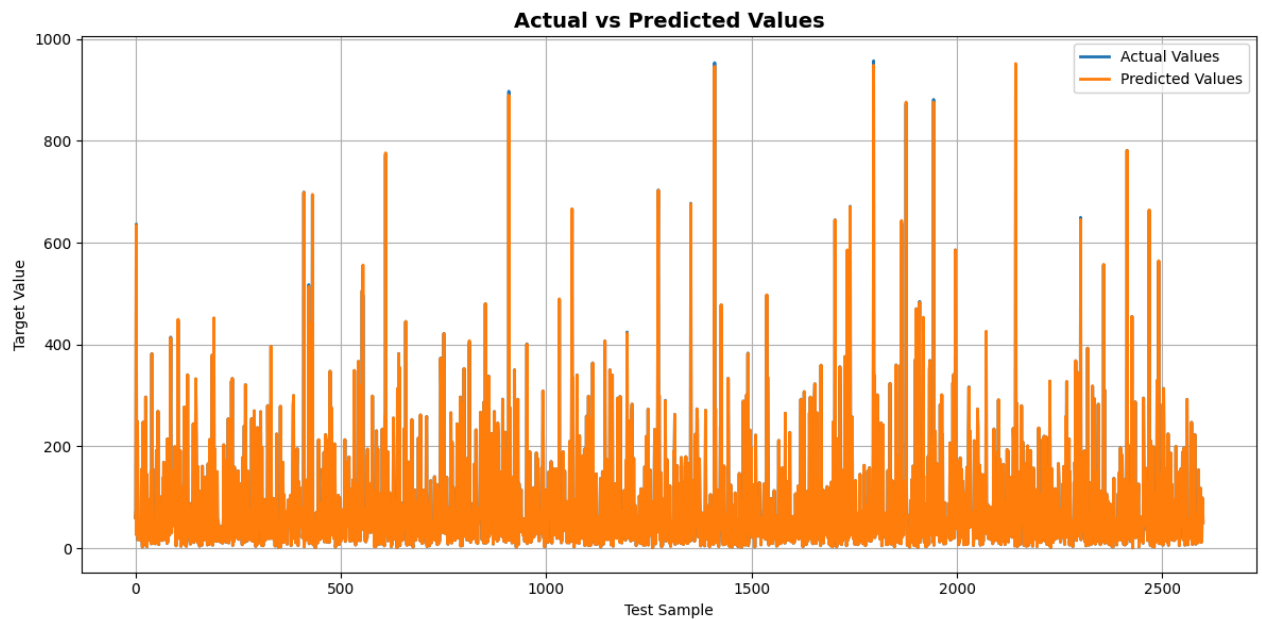
plt.plot(
    y_test.values,
    label="Actual Values",
    linewidth=2
)

plt.plot(
    y_pred,
    label="Predicted Values",
    linewidth=2
)

plt.title(
    "Actual vs Predicted Values",
    fontsize=14,
    fontweight="bold"
)

plt.xlabel("Test Sample")
plt.ylabel("Target Value")

plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



Final Model Performance

```
final_report = pd.DataFrame({
    "Metric": [
        "MAE",
        "MSE",
        "RMSE",
        "R2 Score"
    ],
    "Linear Regression": [
        round(linear_mae, 4),
        round(linear_mse, 4),
        round(linear_rmse, 4),
        round(linear_r2, 4)
    ],
    "Random Forest Regression": [
        round(rf_mae, 4),
        round(rf_mse, 4),
        round(rf_rmse, 4),
        round(rf_r2, 4)
    ]
})

display("FINAL MODEL PERFORMANCE")
display(final_report)
```

'FINAL MODEL PERFORMANCE'

	Metric	Linear Regression	Random Forest Regression
0	MAE	0.0	0.0936
1	MSE	0.0	1.5364
2	RMSE	0.0	1.2395
3	R ² Score	1.0	0.9999

Best Model Selection

```
# Select the model with the highest R2 score
if rf_r2 >= linear_r2:
    best_model_name = "Random Forest Regression"
    best_r2 = rf_r2
else:
    best_model_name = "Linear Regression"
    best_r2 = linear_r2

display("BEST MODEL")

print("Best Model:", best_model_name)
print("Best R2 Score:", round(best_r2, 4))
```

```
'BEST MODEL'
Best Model: Linear Regression
Best R2 Score: 1.0
```

✓ Feature Importance & Model Insights

```
feature_importance = pd.DataFrame({
    "Feature": X_train.columns,
    "Importance": rf_model.feature_importances_
})

# Sort by importance
feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
).reset_index(drop=True)

# Display feature importance
print("FEATURE IMPORTANCE")

display(feature_importance)
```


FEATURE IMPORTANCE

Feature Importance

Feature Importance Plot

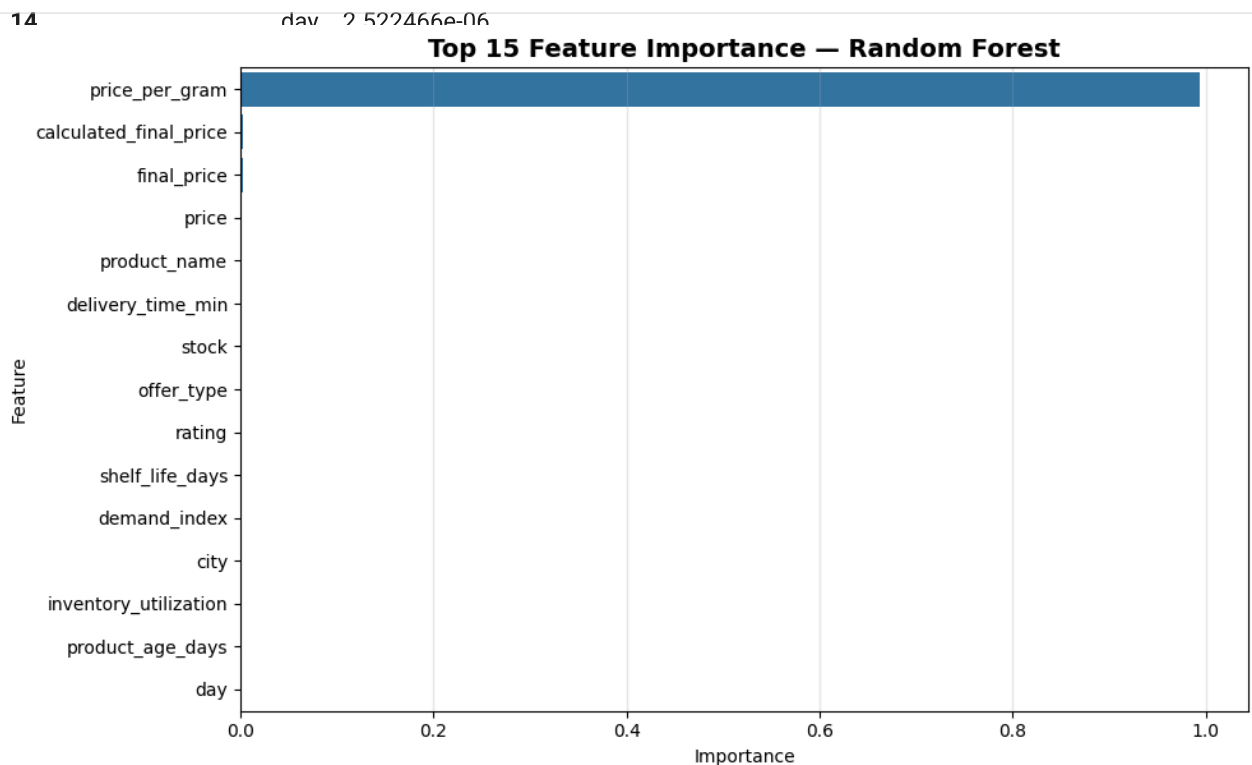
```
1 price_per_gram 9.943139e-01
2 calculated_final_price 2.988183e-03

plt.figure(figsize=(10, 6))

sns.barplot(
    data=feature_importance.head(15),
    x="Importance",
    y="Feature"
)

plt.title(
    "Top 15 Feature Importance – Random Forest",
    fontsize=14,
    fontweight="bold"
)

plt.xlabel("Importance")
plt.ylabel("Feature")
plt.grid(axis="x", alpha=0.3)
plt.tight_layout()
plt.show()
```



Top 10 Important Features

```
29 brand 8.263195e-07
30 is_organic 5.921340e-07
31 delivery_status 5.632162e-07

print("TOP 10 IMPORTANT FEATURES")

display(feature_importance.head(10))

34 category 1.713881e-07
```

TOP 10 IMPORTANT FEATURES		
35	weight_g	3.976036e-08
Feature Importance		
0	price_per_gram	0.994314
1	calculated_final_price	0.002988
2	final_price	0.002597
3	price	0.000026
4	product_name	0.000008
5	delivery_time_min	0.000006
6	stock	0.000005
7	offer_type	0.000004
8	rating	0.000004
9	shelf_life_days	0.000003

Final Model Visualization

